Artificial Intelligence Review (2025) 58:408
https://doi.org/10.1007/s10462-025-11391-8

Cloud-edge-end collaborative caching and UAV-assisted offloading decision based on the fusion of deep reinforcement learning algorithms

Sifeng Zhu[1] · Zhaowei Song[1] · Changlong Huang[1] · Rui Qiao[2] · Hai Zhu[3]

Received: 26 August 2024 / Accepted: 31 August 2025 / Published online: 5 November 2025
© The Author(s) 2025

Abstract
The cloud-edge-end collaboration system provides a new impetus for the development of intelligent transportation. In order to optimize the quality of service for intelligent transportation system users and improve system resource utilization, A three-tier caching strategy for cloud-edge-end collaboration based on efficiency collaboration task popularity (CSEPCA) was proposed, which exploits server resource characteristics and performs fine-grained cache replacements based on real-time task popularity to address the challenges associated with balancing server cache space and cost. To achieve an optimal balance between server cache space and cost, the problem of determining the availability of server cache space is formulated as a constrained markov decision process (CMDP), and an enhanced deep reinforcement learning algorithm based on soft updating (AT-SAC) was designed to achieve multi-objective optimization of system latency, energy consumption, and resource depletion rate, with the aim of improving service response speed and enhancing user service quality. To address challenges in effectively serving vehicles in areas with weak communication signals from cloud-edge servers, UAV swarms were introduced to assist with vehicle task offloading computations. A comprehensive optimization algorithm (Co-DRL-P) was proposed, which integrates enhanced deep reinforcement Learning (ERDDPG) and improved particle swarm optimization (A-PSO) algorithms to optimize UAV trajectories and communication angles, aiming to deliver superior service quality to users. Finally, we evaluate the performance of the proposed scheme through comprehensive simulation experiments. Specifically, when the number of users is 30, the system latency of the proposed scheme is 17.9%, 11.5%, 2.6%, and 60.2% lower than baseline schemes such as DQN, DDPG, TD3, and collaborative randomized schemes, and the system energy consumption is reduced by 20.6%, 15.9%, 9.4%, and 129.9%. Notably, the overall system cost for drone-assisted user offloading is reduced by approximately 49.6% in areas with weak cloud server signals.

Keywords Cloud-edge-end collaboration · Caching policy · Task offloading · Deep reinforcement learning · UAV-assisted offloading

Extended author information available on the last page of the article

1 Introduction

Intelligent transportation systems can provide users with high-quality services and real-time road traffic information, thereby increasing travel efficiency and driving safety(Zhou et al. 2021; Liu et al. 2023). This trend has become the mainstream direction for the future development of the transportation industry. It has emerged as the prevailing trend in the future development of the transportation industry(Dai 2023; Heidari and et al. 2022). As a novel paradigm for addressing intelligent transportation challenges, the cloud-edge-end cooperative system leverages the resource benefits of cloud computing technology and the proximal advantages of edge computing technology. This approach has garnered significant attention in research and development efforts(Lu 2023). However, the rise of compute-intensive applications such as autonomous driving, virtual and augmented reality, and smart internet of things (IoT) has led in-vehicle users to demand a better service experience(Yan et al. 2022). For example, autonomous driving applications require in-vehicle terminals to quickly process application requests to navigate complex traffic scenarios, whereas virtual reality applications impose stringent latency requirements and significant energy consumption demands on the system. This presents a formidable challenge for the intelligent transportation cloud-edge-end collaboration system (Heidari et al. 2023; Wu et al. 2024).

The requirements of intelligent transportation systems for low latency, low energy consumption, and high-quality services are difficult to meet using cloud caching or edge caching alone (Zeng et al. 2023). The cloud caching strategy leverages the substantial resource advantage of cloud servers to cache services in the remote cloud. When a vehicle terminal requests a service response, precached application services can be readily provided. This method, which trades time loss for reduced energy consumption, can partially mitigate the energy consumption of the vehicle terminal. However, it may not be suitable for new latency-sensitive application services. Conversely, the edge caching strategy significantly reduces latency in data transmission by caching applications on nearby edge servers. Edge servers have limited cache space, making it impractical to cache all varieties of application services. Moreover, indiscriminate caching of applications on edge servers leads to significant resource waste, reduces operational efficiency, and may even cause damage to the servers (Chen et al. 2023). Caching undifferentiated applications leads to considerable waste of edge server resources and decreased efficiency and may even result in damage to the edge server. Nevertheless, with the rapid increase in the number of vehicle applications, it has become challenging to cache all types of application services within the limited cache space of edge servers (Long et al. 2023; Wang et al. 2021).

The cloud-edge-end collaborative caching strategy can significantly improve the efficiency of the intelligent transportation system and reduce the system latency and energy consumption (Chai et al. 2023). cloud-edge-end collaborative caching method comprehensively utilizes the cache space of cloud servers, edge servers, and in-vehicle terminals, which can cache a large number of application services to a large extent, and to a certain extent, can satisfy the service requirements of users. However, with the dramatic increase of user scale, the indiscriminate caching strategy not only causes the caching burden of the server to reduce its operational efficiency, but also exacerbates the service burden of in-vehicle terminals to a certain extent. Therefore,in the intelligent transportation cloud-edge-end cooperative computing scenario, comprehensively leveraging the advantages of

cloud servers, edge servers, and vehicle terminals on the basis of the usage characteristics of vehicle applications remains a significant challenge.

In areas with weak signal coverage from cloud-edge servers, unmanned aerial vehicle (UAV) terminals deployed as airborne base stations or signal relay stations can provide effective mission-assisted offloading services to user terminals (Luo et al. 2023). However, the collaborative dynamics and operational trajectory of UAV terminals significantly influence the blocking probability and communication delay within communication system networks (Heidari et al. 2023). Traditional optimization strategies for UAV communication perspectives and trajectories prioritize network coverage and target single-objective constraints such as latency or energy consumption, often ignoring the collaborative dynamics between UAV swarms. While these deployment methods aim to satisfy users' service requests, they may only partially satisfy the system's communication requirements, often resulting in wasted resources due to overlapping signal coverage areas (Wang et al. 2023). In addition, the trajectory optimization of UAV swarm operation is often constrained by the flight trajectory of individual UAVs, which not only affects the endurance of UAVs but also increases the communication delay and energy consumption with the target users. Therefore, well-designed optimization strategies for UAV trajectories and communication angles are used to assist user offloading for intelligent transportation system users, which effectively reduce both the communication delay and energy consumption of the system (Yin et al. 2023).

Several studies have focused on cloud-edge-end cooperative caching strategies and UAV terminal-assisted offloading problems. To mitigate the cumulative cache storage cost within the system, the reference (Hou et al. 2024) introduces a cache space control algorithm that uses edge servers to cache frequently accessed services, thereby maintaining cumulative latency at an acceptable level. Additionally, the reference (Tian 2023) presented a DRL-CA algorithm for cache admission, which aims to reduce computational complexity and communication costs while optimizing cache efficiency. To balance hit and interruption rates, the reference (Zhang 2022) introduced an on-demand adaptive caching algorithm and a fair utility-oriented vehicle edge caching scheme. This scheme adjusts the eviction time of cached content on the basis of the dynamics of requests and content popularity. However, this caching scheme is designed solely for in-vehicle terminals and does not consider the abundant resources of cloud-edge servers. The reference (Zhu et al. 2022) and Yan et al. (2023) explore the task scheduling problem in UAV mobile edge computing systems, aiming to minimize costs and completion times during UAV-assisted task offloading.

In order to maximize the use of limited cache resources in intelligent transportation systems and improve user service quality, in this paper, we formulate the intelligent transportation problem as a multi-objective optimization problem that integrates system latency, energy consumption and resource loss. To improve the service response speed and reduce the system energy consumption, we design a cloud-edge-end collaborative caching strategy based on the characteristics of server caching resources and the attributes of task requests in intelligent transportation scenarios, and propose an enhanced deep reinforcement learning algorithm (AT-SAC) to achieve multi-objective optimization of latency, energy consumption, and resource loss to achieve a fine-grained trade-off between the caching space and cached content. specifically,In scenarios challenged by weak signals in cloud-edge servers, the issue of UAV swarm cooperation to assist in terminal task offloading is explored. By integrating enhanced deep reinforcement learning (ERDDPG) and improved particle swarm

optimization (A-PSO), a comprehensive optimization algorithm (Co-DRL-P) was devised to optimize UAV swarm flight trajectories and communication angles, enabling efficient task offloading by user terminals even in areas with weak cloud server signals.

The contributions of this paper can be outlined as follows:

1. We design a four-layer communication architecture for telematics with cloud-edge-terminal-UAVs collaboration, and use a multi-objective optimization approach to achieve the optimal trade-off between latency, energy consumption, and system resource loss in intelligent transportation systems.
2. Taking into account the distinct resource characteristics of cloud servers, edge servers, and in-vehicle terminals, as well as user usage habits, we propose a cloud-edge-end collaborative three-tier caching strategy (CSEPCA), which leverages the popularity of efficiency collaboration tasks to balance the relationship between task popularity, user usage patterns, and server resources.
3. We combine the improved deep-reinforcement learning DDPG algorithm (ERDDPG) with the enhanced Particle Swarm Optimization algorithm (A-PSO) to design a fusion algorithm, called Co-DRL-P. This algorithm optimizes system latency, energy consumption, and UAV endurance as primary objectives and constraints, while simultaneously optimizing the UAV swarm trajectory and communication angles, with the goal of maximizing assistance to the on-board user in task offloading.
4. Finally, we evaluate the performance of the proposed scheme through comprehensive simulation experiments. Specifically, when the number of users is 30, the system latency of the proposed scheme is 17. 9%, 11. 5%, 2. 6%, and 60. 2% lower than the baseline schemes such as DQN, DDPG, TD3 and collaborative randomized schemes, and the system energy consumption is reduced by 20.6%, 15.9%, 9.4%, and 129.9%. In particular, the total system cost for drone-assisted user offloading is reduced by approximately 49.6% in areas with weak cloud server signals.

2 Related work

Currently, research on the cloud-edge-end collaboration problem within intelligent transportation has reached a significant scale (Jiang et al. 2023; Heidari et al. 2023; Peng et al. 2024; Wu et al. 2024). The research focus primarily encompasses the decision to offload onboard tasks, resource allocation, minimizing system delay, reducing energy consumption, and optimizing caching strategies (Heidari et al. 2023; Xia et al. 2023). The reference (Chen et al. 2023) developed a distributed multi-hop task offloading decision model and demonstrated that the proposed method exhibits good latency performance for a varying number of tasks by means of greedy and discrete bat algorithms. The reference (Deng et al. 2023) describes an autonomous partial offloading system for latency-sensitive computational tasks in a multi-user environment and experimentally demonstrates the effectiveness of the proposed scheme. Reference (Dai 2023) proposes a cloud-assisted fog computing framework that integrates task offloading and service caching, introduces a distributed task offloading algorithm based on non-cooperative game theory, and integrates system latency and energy consumption according to dynamic service caching conditions. Reference (Yu et al. 2023) proposed a novel multi-agent information broadcasting and judgment algorithm to

collaboratively allocate resources and optimize various goals, including latency and energy consumption.

In the environment of intelligent transportation systems, meeting the diverse service needs of users requires the rational allocation of various resources and the full utilization of the advantages offered by different types of servers (Dai et al. 2023). A well-designed caching strategy can effectively reduce system energy consumption, markedly enhance task calculation offloading speed, and ultimately enhance system efficiency. However, the intertwined relationship between the computing resources and caching resources of edge servers poses a challenge for servers to strike a balance between computation offloading strategies and caching strategies (Tian 2023). Relying solely on a single edge caching strategy may not adequately meet the demands of a connected vehicle intelligent transportation system. The reference (Zhang et al. 2022)introduces the DRL-CA algorithm for edge cache admission, accompanied by a federated learning-based parameter sharing mechanism to alleviate signaling overhead during collaboration. Additionally, In the reference (Li 2023), a learning-based collaborative caching strategy (LECS) was designed to accurately estimate content hotness by constructing a time-evolving network-driven content hotness prediction model, which introduces the notion of content cache value to evaluate the importance of cached content on a particular edge server.

Research on UAV-assisted optimization of resources in intelligent transportation systems has garnered significant attention in recent years (Chen et al. 2024; Andreou et al. 2023). The literature (Yang and Yao 2023) studied the resource allocation and control problem of UAV-assisted IoT communication devices, and constructed an IoT system incorporating UAVs by constructing a UAV-assisted IoT resource allocation and control model to satisfy the user's communication experience while allocating channel resources. The literature (Zhao et al. 2024) explores the integration of UAVs with reconfigurable intelligent surfaces to optimize communication resources in intelligent transportation systems. The study proposes a strategy for real-time adjustment of UAV trajectories and RIS phase shifts, aiming to improve position prediction for communicating vehicles in dynamic environments. The literature (Miao et al. 2024) proposes a secure and effective authentication protocol for drone-assisted vehicular networking that utilizes elliptic curve cryptography to ensure the security of authentication, which is resistant to known attacks and has a superiority in terms of overhead.

The existing literature has investigated collaborative offloading and caching for cloud-edge-end servers, offering certain practical insights. However, there is a lack of in-depth exploration regarding the potential system benefits of designing specialized caching methods for different servers. Tailoring caching strategies to leverage the unique advantages of specific servers could significantly enhance system service efficiency and improve user experiences. Furthermore, in the research on offloading strategies for UAV-assisted cloud-edge collaboration scenarios, UAVs are primarily considered as communication relays or individual service providers. This approach severs the interconnected nature of UAV groups and fails to fully capture their collective performance gains, thereby missing opportunities to maximize the strategic effectiveness of UAV swarms as a cohesive communication unit.

In contrast to the existing literature, in this document a service caching and replacement method has been designed to accommodate the unique characteristics of cloud servers, edge servers and terminals in vehicles within intelligent transportation scenarios. By accounting for the specific advantages of each server type, a three-tier caching policy for cloud-edge-

end collaboration based on the popularity of collaborative tasks (CSEPCA) was proposed, and the AT-SAC algorithm was employed to achieve multi-objective optimization across system delay, energy consumption, and resource utilization. Additionally, in the context of UAV-assisted user offloading within cloud-edge-end collaboration scenarios, a focus has been placed on the cohesive coordination among UAV groups. An improved particle swarm optimization algorithm (A-PSO) and an enhanced deep deterministic policy gradient algorithm (ERDDPG) were integrated to develop a comprehensive optimization algorithm (Co-DRL-P), allowing for the optimization of UAV group flight trajectories and communication angles. This ensures that user terminals maintain efficient task offloading capabilities, even in areas with weak cloud-edge server signals.

3 Problem modeling

3.1 System model

This paper presents an in-depth analysis of a 3D spatial traffic scenario involving a smart connected vehicle navigating a smart city road. In this scenario, the primary task is to gather traffic flow information to assist in smart driving. The system setup includes two types of roads: a 3 km two-way urban road and a 2 km two-way suburban road (an area with weak signal coverage for cloud-edge servers). In this setting, edge servers with roadside units are strategically placed on both sides of the urban road, ensuring strong signal coverage. The central cloud server is located in the city's cloud infrastructure, while multiple UAV terminals fly overhead at a fixed speed. Additionally, vehicle-mounted signal sensors, along with other sensing devices such as traffic signals, cameras, and radars located along the road, provide real-time data on traffic conditions and navigation information. The collected traffic data and road condition information are then processed using artificial intelligence and big data technologies, transforming the raw information into the required traffic flow data for the system. To more effectively implement the UAV-assisted intelligent transportation scenario proposed in this paper, we make the following assumptions regarding the model:

- In this study, we assume that the communication coverage of the cloud server extends across the entire urban area of the smart city, enabling it to offer a wide range of computation and communication services to devices throughout the city. The communication coverage of the edge server is confined to a circular area with a radius of 1.5 kms, primarily serving devices located in urban transportation hubs or critical areas, providing low-latency computing and data transmission. The communication range of drones is relatively small, limited to a circular area with a radius of 100 m, suitable for tasks in specific localized zones. In contrast, the communication range of smart grid-connected vehicle terminals extends to a radius of 10 m, mainly supporting localized communication between vehicle devices. The communication radius of different devices influences the frequency of interaction and the data transmission delay. A smaller communication range may require more frequent communication between devices, whereas a larger range could present challenges related to resource scheduling and communication interference.
- We assume that task requests from vehicle terminals in the intelligent transportation system follow a Poisson distribution. The arrival times of these task requests are random

and independent, and the task arrival rate remains constant over time. This assumption is effective in capturing the inherent randomness and independence of task requests within the system. While it is true that, in real-world scenarios, the arrival rate of task requests may fluctuate due to factors such as traffic flow, weather conditions, and other external influences, we adopt the Poisson distribution as the foundational model. This choice simplifies the analysis and helps to highlight the fundamental statistical properties of task request arrivals.

- The small-scale fading channel gain of the wireless communication link is consistent with the first-order autoregressive model of this paper.
- The delay incurred during the vehicle task's computational offloading cycle primarily includes both the computational and transmission delays. Energy consumption is considered only in terms of the computational and transmission energy required for the task, without accounting for the delays and energy losses associated with the engineering of the wired communication method.

The system model designed in this paper is shown in Fig. 1.

The intelligent transportation cloud-edge-end-UAVs communication framework designed in this paper consists of a central cloud server, N edge servers, M vehicle-mounted terminals (each terminal handles I structure-intensive tasks), and K UAVs to form a drone swarm. Where, the set of edge servers is $ES = \{ecd_1, ecd_2, \ldots, ecd_N\}, n \in [1, N]$, the set of vehicle terminals is $CR = \{car_1, car_2, \ldots, car_M\}, m \in [1, M]$, the set of UAVs is $UA = \{uav_1, uav_2, \ldots, uav_K\}, k \in [1, K]$ and the set of tasks is $TK = \{task_1, task_2, \ldots, task_I\}, i \in [1, I]$, $v_{uav_k}^{tran} = (v_{k1}, v_{k2}, \ldots, v_{kD})$is the set of velocity parameters of the UAVs, $loc_{uav_k}^{x,y} = (x_{k1}, x_{k2}, \ldots, x_{kD})$ is the set of position parameters of the UAVs.

In this framework, each communication entity within the intelligent transportation system is abstracted as a mathematical model. The representation and parameter patterns of the model are detailed in Table 1.

3.2 Cloud-edge-end collaborative caching model

Effective caching strategies can increase the operational efficiency and task responsiveness of intelligent transportation systems (Zhou et al. 2023). Considering the diversity of caching resources among various servers and aiming to fully leverage the resource advantages of different servers, this paper proposes a three-tier vehicle networking caching model featuring cloud-edge-end collaboration. Specifically, in this paper, we design a centralized cloud caching policy on the basis of the popularity of tasks on the basis of the resource advantage of cloud services, an edge caching policy with the most frequently used cache for tasks and the longest unused replacement for tasks on the basis of the proximity advantage of edge servers, and an onboard local caching policy with the most recently used cache for tasks and the least used replacement for tasks on the basis of the resources of vehicle terminals.

In the process of vehicle driving, considering the efficiency of service request completion and the limited resources of the vehicle itself, the user terminal offloads part of the task to the cloud server or edge server for computation. The introduction of the caching strategy greatly reduces delay and energy consumption in this process. The cloud-edge-end collaborative caching strategy designed in this paper is based on the caching status of tasks

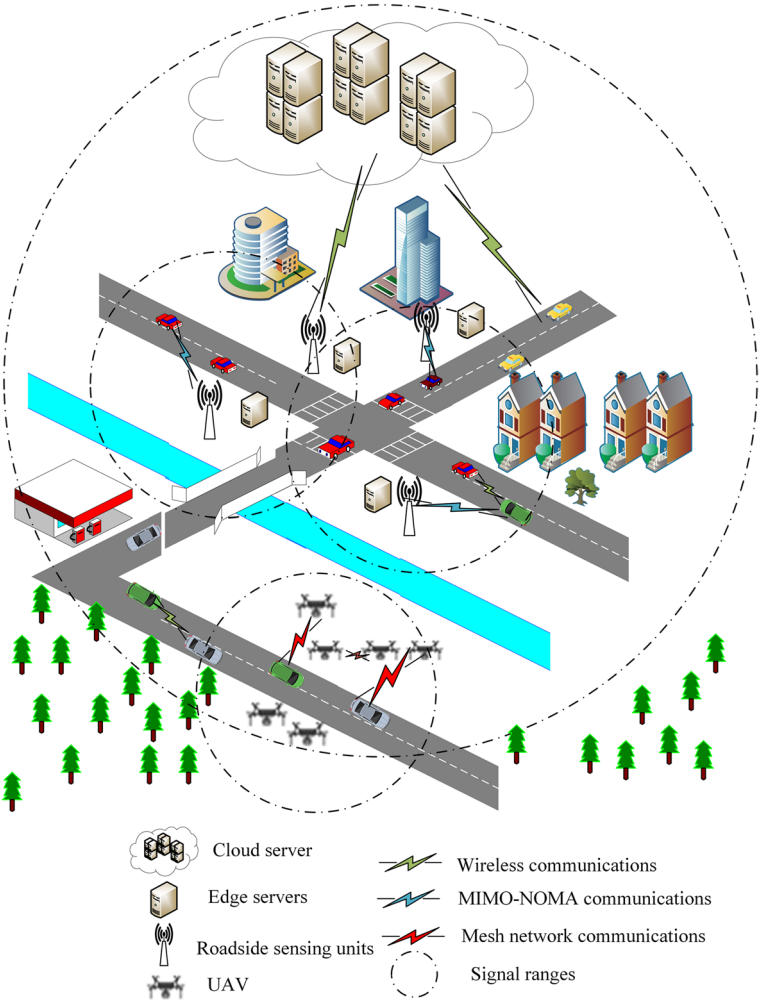


Fig. 1 System model

in the cloud-edge-end caching matrix. This strategy evaluates overall consumption during task offloading and overloading, using normalized overall consumption as a reward function for the deep reinforcement learning process. This approach achieves an optimal trade-off in cloud-edge-end caching. Finally, the deep reinforcement learning algorithm is used to attain the optimal balance for cloud-edge-end caching.

The cloud-edge-end collaborative caching model designed in this paper is shown in Fig. 2.

3.2.1 Cloud cache

While the ample resources of cloud servers can mitigate the constraints imposed by the limited resources of edge servers and vehicle terminals, adopting an undifferentiated cloud

Table 1 Model parameters and their implications

Model representations	Symbol implications
f_{cloud}^{calc}	Computing resources of cloud server
s_{cloud}^{cach}	Caching resources of cloud server
p_{cloud}^{calc}	Computational power of cloud server
p_{cloud}^{tran}	Transmission power of cloud server
$f_{ecd_n}^{calc}$	Computing resources of ecd_n
$s_{ecd_n}^{cach}$	Caching resources of ecd_n
$p_{ecd_n}^{calc}$	Computational power of ecd_n
$p_{ecd_n}^{tran}$	Transmission power of ecd_n
$f_{uav_k}^{calc}$	Computing resources of uav_k
$p_{uav_k}^{calc}$	Computational power of uav_k
$p_{uav_k}^{tran}$	Transmission power of uav_k
$f_{car_m}^{calc}$	Computing resources of car_m
$s_{car_m}^{cach}$	Caching resources of car_m
$p_{car_m}^{calc}$	Computational power of car_m
$p_{car_m}^{tran}$	Transmission power of car_m
$v_{car_m}^{tran}$	The velocity parameters of car_m
$loc_{car_m}^{x,y}$	The position parameters of car_m
$d_{task_i}^{dav}$	The size of the data volume of $task_i$
$f_{task_i}^{calc}$	Required computing resources of $task_i$
$s_{task_i}^{cach}$	Required caching resources of $task_i$

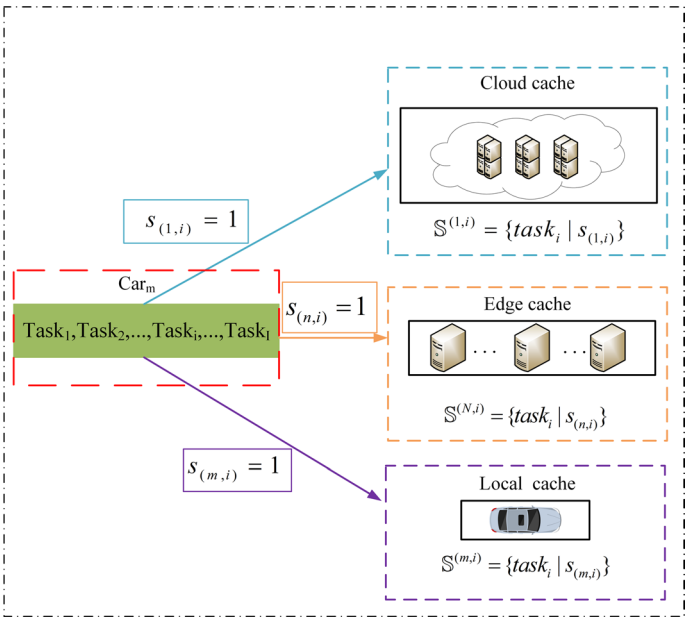


Fig. 2 Cloud-edge-end collaborative caching model

caching approach may result in low operational efficiency of cloud servers due to excessive resource waste (Zhou et al. 2023). In this work, we design a centralized send-and-receive cloud caching strategy on the basis of the global popularity of tasks and use the caching matrix $\mathbb{S}^{(1,i)} = \{task_i \mid s_{(1,i)}\}$ to indicate the caching situation of the central cloud server, where $s_{(1,i)} = 1$ indicates that $task_i$ is cached in the central cloud server.

The popularity of a task is quantified by its global request probability per unit of caching time, with a higher request probability indicating greater popularity. Tasks are cached centrally on the cloud server in descending order of request probabilities, from highest to lowest (Yuan et al. 2023). The cloud server centrally caches tasks on the basis of the request probability, prioritizing tasks in descending order from largest to smallest probability. Let $task_i$ be ranked i in the queue of all request tasks of the cloud server at cache time t_0. From the distribution of *Zipf*, the request probability P_{task_i} of $task_i$ in unit cache time is shown in Eq. (1).

$$P_{task_i} = \frac{i^{-a}}{\sum_{i=1}^{n}(i^{-a})}, \quad i \in n \tag{1}$$

where $a(a > 0)$ indicates the degree of concentration of requests to $task_i$, and a larger a indicates more concentrated requests. n is the total number of tasks at t_0.

3.2.2 Edge cache

In this paper, we leverage the proximity advantage of edge servers and their ample caching resources to propose an edge caching policy. This policy is based on caching the most frequently accessed tasks and replacing those that have remained unused for the longest period. The caching weight of a task is represented by the edge caching evaluation function $F(task_i)$, and a high value of the evaluation function indicates that the task is cached with a higher weight. The caching matrix $\mathbb{S}^{(N,i)} = \{task_i \mid s_{(n,i)}\}$ denotes the caching status of ecd_n, and when $s_{(n,i)} = 1$ denotes that $task_i$ is cached in ecd_n.

Let the total number of times that the vehicle terminal requests $task_i$ from the edge server at t_0 be $S(task_i)$. $T(task_i)$ be the last access moment of $task_i$. Then, the edge caching function $F(task_i)$ is shown in Eq. (2).

$$F(task_i) = w \times S(task_i) - (1 - w) \times T(task_i) \tag{2}$$

where $w(w \in (0, 1])$ is the weight parameter used to balance the access frequency and access time.

3.2.3 Local cache

In this paper, we devise a vehicle local caching strategy that prioritizes recent-use caching of tasks and least-use replacement of tasks, considering the limited resources available in vehicle terminals. The local caching function $F_0(task_i)$ is used to represent the caching weight of a task, and a higher value of the evaluation function indicates that the task is cached with

greater weight. The caching matrix $\mathbb{S}^{(m,i)} = \{task_i \mid s_{(m,i)}\}$ denotes the caching situation of car_m, when $s_{(m,i)} = 1$ denotes that $task_i$ is cached in car_m.

Let $t_0(task_i)$ be the last access moment of $task_i$ and let $d^m_{arr}(t_0)$ be the task arrival of car_m at time slot t_0; then, the length of the cache queue $L^m(t_0+1)$ at time slot (t_0+1) is shown in Eq. (3).

$$L(t_0+1) = \min\left(\max\left[L^m(t_0) - d^m_{arr}(t_0)\right], L^m_{\max}\right) \tag{3}$$

where $L^m(t_0)$ is the cache queue length of car_m at time slot t_0 and where $L^m_{\max}$ is the maximum cache queue length of car_m.

The local cache evaluation function $F_0(task_i)$ is shown in Eq. (4).

$$F_0(task_i) = e^{-[t_0 - T_0(task_i)\times\mathcal{E}]} \tag{4}$$

where $\mathcal{E}$ represents the decay constant, which serves as an indicator of the sensitivity of time differences. A smaller decay constant implies greater sensitivity to changes in time.

3.3 Communication model

This paper assumes that the central cloud server can offer all types of services for the service request units. Smart networked vehicles, constrained by limited resources, can manage only a small number of application services. Drone terminals are capable of providing auxiliary offloading services for smart networked vehicle terminals. Additionally, edge servers are only capable of servicing vehicle terminals within their respective ranges. The edge server serves only vehicle terminals within its operational range (Yuan et al. 2023; Araf et al. 2023).

The communication model designed in this paper is shown in Fig. 3.

The time when the smart networked vehicle terminal passes through the edge server is divided into $\mathcal{T}(\tau = \{1, 2, 3, t_0, \ldots, T\})$ time slots, and the length of each time slot is t_0. Assuming that the channel state follows quasi-static flat Rayleigh fading, the channel state in neighboring time slots is kept free of interference and time-varying when wireless communication takes place between servers in the same time slot. Let the channel gain matrix of the mission vehicle car_m in time slot t_0 be $h_m(t) \in \mathbb{C}^{k\times 1}$, then, the data transfer rate between the mission vehicle car_m and the edge server ecd_n via the V2E communication technique is shown in Eq. (5).

$$R^{m,n}_{V2E} = \frac{B^{m,n}_{ecd_n}}{\text{channel}_{V2E}} \log_2\left(1 + \frac{p^{tran}_{car_m} \times h_m(t_0)}{\theta^2}\right) \tag{5}$$

where $B^{m,n}_{ecd_n}$ is the communication bandwidth between car_m and ecd_n, channel_{V2E} is the number of communication channels between car_m and θ^2 denotes the additive Gaussian white noise power.

The channel gain $h_m(t_0)$ between car_m and ecd_n is shown in Eq. (6).

$$h_m(t_0) = h^a_m(t)\sqrt{\frac{\alpha}{v^{tran}_{car_m} \times t_0}} \tag{6}$$

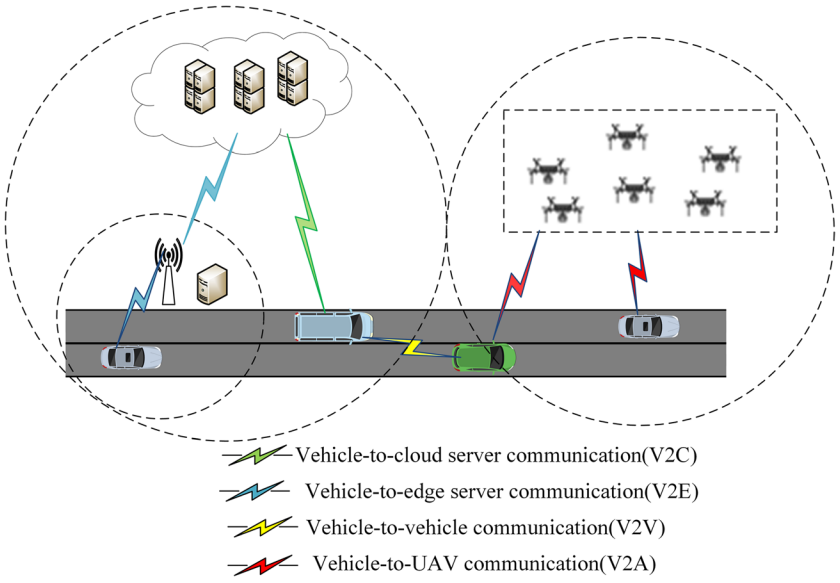


Fig. 3 Communications model

To make the communication model more in line with the real traffic situation, this paper introduces a time-dependent first-order autoregressive model to simulate the small-scale fading channel gain of the wireless communication link in different time slots t_0, $h_m^a(t_0+1)$ as shown in Eq. (7).

$$h_m^a(t_0+1) = \beta h_m^a(t_0) + \sqrt{1-\beta^2}b(t_0+1) \tag{7}$$

where $\beta(\beta = J_0(2\pi f_m t_0))$ is the correlation coefficient between neighboring time slots in the *Jakes* model, $b(t_0+1)$ are error vectors that obey a Gaussian distribution, and the Doppler frequency deviation of car_m due to its movement f_m is shown in Eq. (8).

$$f_m = \frac{v_{car_m}^{tran}}{\lambda} \tag{8}$$

where λ is the signal wavelength of the mission vehicle during data transmission.

According to the Shannon formula, the data transfer rates between car_m and the central cloud server, UAV terminals, and other vehicle-mounted terminals are shown in Eqs. (9), (10), and (11).

$$R_{V2C} = \frac{B_0}{\text{channel}_{V2C}} \log_2\left(1 + \frac{s_0 \times p_{car_m}^{tran}}{\theta^2}\right) \tag{9}$$

$$R_{V2A} = \frac{B_{uav}}{\text{channel}_{V2A}} \log_2\left(1 + \frac{s_0 \times p_{car_m}^{tran}}{\theta^2}\right) \tag{10}$$

$$R_{V2V} = \frac{B_1}{\text{channel}_{V2V}} \log_2 \left(1 + \frac{s_0 \times p_{car_m}^{tran}}{\theta^2}\right) \tag{11}$$

where B_0 is the communication bandwidth between car_m and the central cloud server, B_{uav} is the communication bandwidth between car_m and the UAV, B_1 is the communication bandwidth between car_m and other vehicle terminals, channel_{V2C} is the number of communication channels between car_m and the central cloud server, $channel_{V2A}$ is the number of communication channels between car_m and the UAV terminal, $channel_{V2V}$ is the number of communication channels between car_m and the other vehicle terminals, and s_0 is a channel loss constant.

3.4 Delay modeling and energy consumption modeling

In general, servers with greater computational capabilities execute tasks more rapidly. Considering this paper's focus on processing computationally intensive tasks, the latency incurred by on-board tasks during the computational offloading cycle primarily comprises two components: computational latency and transmission latency (Samanta et al. 2023).

3.4.1 Latency and energy consumption in local computing scenarios

At time slot t_0, the delay $t_{task_i}^{exe}$ when task_i is computed locally through the vehicle is shown in Eq. (12).

$$t_{task_i}^{exe} = \begin{cases} \frac{f_{task_i}^{calc}}{f_{car_m}^{calc}}, & s_{(m,i)} = 0; \\ 0, & s_{(m,i)} = 1; \end{cases} \tag{12}$$

where the binary variable $s_{(m,i)}$ represents the local cache of task_i, $s_{(m,i)} = 0$ represents that task_i is not cached locally, and $s_{(m,i)} = 1$ represents the locally cached task_i.

The energy consumption is calculated when task_i is executed locally, as shown in Eq. (13).

$$e_{task_i}^{exe} = \begin{cases} \frac{f_{task_i}^{calc} \times p_{car_m}^{calc}}{f_{car_m}^{calc}}, & s_{(m,i)} = 0; \\ 0, & s_{(m,i)} = 1; \end{cases} \tag{13}$$

When task_i completes cross-region transmission computations with the help of other vehicle terminals, in addition to the task transmission with the help of the cooperative edge server, it can also send the data to other vehicles via V2V communication technology because this process needs to occupy the resources of other vehicles. Taking into account the private nature of the resources of other users, this paper considers only the case in which other vehicles have a cache for task_i. When the data are transmitted, V2V communication technology is used. The computational delay of task_i is shown in the following Eq. (14).

$$t_{task_i}^{exe} = \frac{d_{task_i}^{dav} \times \theta_{mn} \times \rho}{R_{V2V}} \tag{14}$$

where the binary variable θ_{mn} indicates whether the task was transmitted via the V2V communication technique, $\theta_{mn} = 1$ indicates that the task was transmitted via the V2V communication technique, and ρ indicates the number of times it was transmitted via the V2V communication technique.

The energy consumption of fort $task_i$ transmission via V2V communication technology is expressed in Eq. (15).

$$e_{task_i}^{exe} = \frac{d_{task_i}^{dav} \times \theta_{mn} \times \rho \times p_{car_m}^{tran}}{R_{V2V}} \tag{15}$$

3.4.2 Latency and energy consumption in edge computing scenarios

At time slot t_0, $task_i$ is executed at ecd_n with a delay that incorporates both the edge-to-end transmission delay and the delay computed by $task_i$ at ecd_n. The execution delay is represented in Eq. (16).

$$t_{task_i}^{exe} = \begin{cases} \frac{d_{task_i}^{dav}}{R_{V2E}^{m,n}} + \frac{f_{task_i}^{calc}}{f_{ecd_n}^{calc}} \sum_{n=1}^{N} s_{(n,i)}, & = 0; \\ 0, & \sum_{n=1}^{N} s_{(n,i)} = 1; \end{cases} \tag{16}$$

The energy consumption for $task_i$ execution at ecd_n is shown in Eq. (17).

$$e_{task_i}^{exe} = \begin{cases} \frac{d_{task_i}^{dav}}{R_{V2E}^{m,n}} \times p_{car_m}^{tran} + \frac{f_{task_i}^{calc}}{f_{ecd_n}^{calc}} \times p_{ecd_n}^{calc} \sum_{n=1}^{N} s_{(n,i)}, & = 0; \\ 0, & \sum_{n=1}^{N} s_{(n,i)} = 1; \end{cases} \tag{17}$$

3.4.3 Latency and energy consumption in cloud server computing scenarios

At time slot t_0, the delay of $task_i$ execution at the cloud server includes the cloud-terminal transmission delay and $task_i$ computation delay at the cloud server. The execution delay $t_{task_i}^{exe}$ is shown in Eq. (18).

$$t_{task_i}^{exe} = \begin{cases} \frac{d_{task_i}^{dav}}{R_{V2C}^{m,n}} + \frac{f_{task_i}^{calc}}{f_{cloud}^{calc}} \times s_{(1,i)}, & = 0; \\ 0, & s_{(1,i)} = 1; \end{cases} \tag{18}$$

The binary variable $s_{(1,i)}$ indicates that $task_i$ is cached in the cloud server. $s_{(1,i)} = 1$ indicates that $task_i$ is cached in the cloud server, and $s_{(1,i)} = 0$ indicates that $task_i$ is not cached in the cloud server. The energy consumption for $task_i$ execution at the cloud server is shown in Eq. (19).

$$e_{task_i}^{exe} = \begin{cases} \frac{d_{task_i}^{dav}}{R_{V2C}} \times p_{car_m}^{tran} + \frac{f_{task_i}^{calc}}{f_{cloud}^{calc}} \times p_{cloud}^{calc} \times s_{(1,i)}, & = 0; \\ 0, & s_{(1,i)} = 1; \end{cases} \tag{19}$$

3.4.4 Delay and energy consumption in UAV-assisted computational scenarios

The delay of $task_i$ execution at the UAV at time slot t_0, includes the transmission delay from car_m to uav_k and the computation delay of the task at uav_k. The execution delay $t_{task_i}^{exe}$ is shown in Eq. (20).

$$t_{task_i}^{exe} = \frac{d_{task_i}^{dav}}{R_{V2A}} + \frac{f_{task_i}^{calc}}{f_{uav_k}^{calc}} \tag{20}$$

The energy consumption for $task_i$ execution at uav_k is shown in Eq. (21).

$$e_{task_i}^{exe} = \frac{d_{task_i}^{dav}}{R_{V2A}} \times p_{car_m}^{tran} + \frac{f_{task_i}^{calc}}{f_{uav_k}^{calc}} \times p_{uav_k}^{calc} \tag{21}$$

The total task offloading delay $T(X)$ and the total energy consumption $E(X)$ are shown in Eqs. (22) and (23).

$$T(X) = \sum_{i=1}^{I} t_{task_i}^{exe} \tag{22}$$

$$E(X) = \sum_{i=1}^{I} e_{task_i}^{exe} \tag{23}$$

3.5 System resource loss evaluation model

The degree of resource depletion reflects the system's efficiency to some extent (Wu 2024). To further optimize the system's resource utilization, this paper proposes a system resource loss evaluation model that integrates the utilization of each server resource with the completion of on-board tasks. It is assumed that the on-board tasks can complete the computation offloading under the maximum tolerable delay $r_{task_i}^{rest}$. The binary variable O_{task_i} is used to denote the output state of $task_i$, and its state representation is shown in Eq. (24).

$$O_{task_i} = \begin{cases} 0, & \text{Output incomplete;} \\ 1, & \text{Output completed;} \end{cases} \tag{24}$$

The resource loss for calculating $task_i$ is shown in Eq. (25).

$$RL_{task_i} = \frac{(1 - O_{task_i}) \times f_{task_i}^{calc}}{\left[(t_{task_i}^{rest} - f_{task_i}^{calc} \times w_{task_i})/(f \times f_d)\right] \times mcl} \tag{25}$$

where f is the size of the computational resources used to compute $task_i$, w_{task_i} is the weighting factor associated with the task, f_d is the size of the computational frequency of the target server, and $mcl = \log_2(1 + snr)$ and snr are the sizes of the signal-to-noise

ratios. In this paper, Eq. (26) delineates the utilization of the system resource loss evaluation model, which gauges the overall resource loss of the system.

$$RL(X) = \sum_{i=1}^{I} RL_{task_i} \tag{26}$$

3.6 Description of the problem

In this paper, we introduce UAV to assist the task offloading computation in the cloud-edge-end collaborative computing scenario of intelligent transportation and design a cloud-edge-end collaborative three-layer caching strategy for vehicular networking with the main objectives of system delay minimization, energy consumption minimization, and system resource loss.

The constructed model of the multi-objective optimization problem is shown in the following Eq. (27).

$$\min T, \min E, \min RL \tag{27}$$

s.t.

$$\begin{aligned}
&\text{C1:} \quad s^{cach}_{s(n,i)} \times task_i \le s^{cach}_{ecd_n}, \quad \forall n \in N, \forall i \in I \\
&\text{C2:} \quad s^{cach}_{s(m,i)} \times task_i \le s^{cach}_{car_m}, \quad \forall n \in N, \forall i \in I, \forall m \in M \\
&\text{C3:} \quad f^{calc}_{task_i} \le f^{calc}_{car_m}, \quad \forall n \in N, \forall i \in I, \forall m \in M \\
&\text{C4:} \quad f^{calc}_{task_i} \le f^{calc}_{ecd_n}, \quad \forall n \in N, \forall i \in I \\
&\text{C5:} \quad f^{calc}_{task_i} \le f^{calc}_{uav_k}, \quad \forall n \in N, \forall i \in I, \forall k \in K \\
&\text{C6:} \quad t^{rest}_{task_i} \le \ell, \quad \forall i \in I \\
&\text{C7:} \quad t^{tran}_{uav_k} \le t^{exe}_{uav_k}, \quad \forall k \in K \\
&\text{C8:} \quad e^{tran}_{uav_k} \le e^{exe}_{uav_k}, \quad \forall k \in K
\end{aligned}$$

Among them, the constraint C1 means that the cache resources required for caching any task cannot exceed the maximum storage resources of the edge server; the constraint C2 means that the cache resources required for caching any task cannot exceed the maximum storage resources of the vehicle terminal; the constraint C3 means that the computing resources for calculating any task cannot exceed the maximum computational resources of the edge server; the constraint C4 means that the computing resources for calculating any task cannot exceed the maximum computational resources of the vehicle terminal; The constraint C5 indicates that the computing resources of any task cannot exceed the maximum computing resources of the UAV terminal; the constraint C6 indicates that $task_i$ can finish the computation offloading within the maximum response delay ℓ; the constraint denotes C7 that the transmission delay of any UAV terminal communication does not exceed the offload delay; and the constraint C8 denotes that the transmission energy consumption of any UAV terminal communication does not exceed the mission offload energy consumption.

4 Solutions

In this work, a distributed parallel cloud-edge-end cooperative caching strategy was designed on the basis of the dynamic, time-varying, and continuous action characteristics of the cloud-edge-end cooperative vehicle networking model (Xiao et al. 2023). The strategy uses a deep reinforcement learning AT-SAC algorithm, and the complexity of the algorithm is analyzed. UAVs swarm collaboration tasks inherently involve the coordination and optimization of multiple agents. PSO demonstrates strong global search capabilities and flexibility, making it highly effective in optimizing the flight trajectories and communication angles of UAVs swarms. This characteristic highlights its unique advantage in assisting users with task offloading and makes it particularly suitable for swarm collaboration optimization scenarios. On the other hand, the DDPG offers a high degree of flexibility and efficiency, excelling in task allocation and resource scheduling under conditions of uncertainty and within a three-dimensional state space. In resource-constrained UAV environments, the enhanced DDPG algorithm features a simpler neural network structure that consumes fewer system resources compared to SAC. Consequently, this paper adopts a combination of the improved PSO algorithm and the ERDDPG algorithm to optimize the UAV offloading strategy for UAV-assisted tasks. Specifically, the improved PSO algorithm is initially applied to optimize the mission offloading strategy and the deployment distribution of UAVs. Second, considering that high-altitude communication is often affected by other communication noise and that the communication and flight angle of the UAV terminal is a continuous quantity between $\left[0, \frac{\pi}{2}\right]$ and many other complex factors, this paper adopts the improved DDPG algorithm to realize the optimal solution for the communication and flight angle of the UAV.

4.1 Collaborative scenarios for optimization strategies

In this section, we subdivide the UAV-assisted cloud-edge-end collaboration scenarios into two sub-scenarios: smart city areas with good communication signals and areas with weak cloud-edge server signals. Among them, in the smart city area with good signal, we use the AT-SAC to provide decision making, and in the cloud-edge server weak signal area we use Co-DRL-P to provide decision making for the system.

The collaboration process of the UAV-assisted cloud-edge-end collaboration system designed in this paper is shown in Fig. 4.

4.2 Optimization scheme based on AT-SAC

4.2.1 AT-SAC algorithm design

The SAC algorithm is an online deep reinforcement learning algorithm with high sample efficiency and high stability (Wu 2024) (Figs. 5, 6, 7). Compared with other reinforcement learning algorithms, the entropy network increases the exploration ability of the environment by maximizing the entropy of the strategy, helps the smart body improve the understanding of the environment, and increases the convergence speed of the understanding. In addition, to improve the stability of the algorithm, the algorithm provides neural networks such as the policy network and the value function network, and the introduction of the

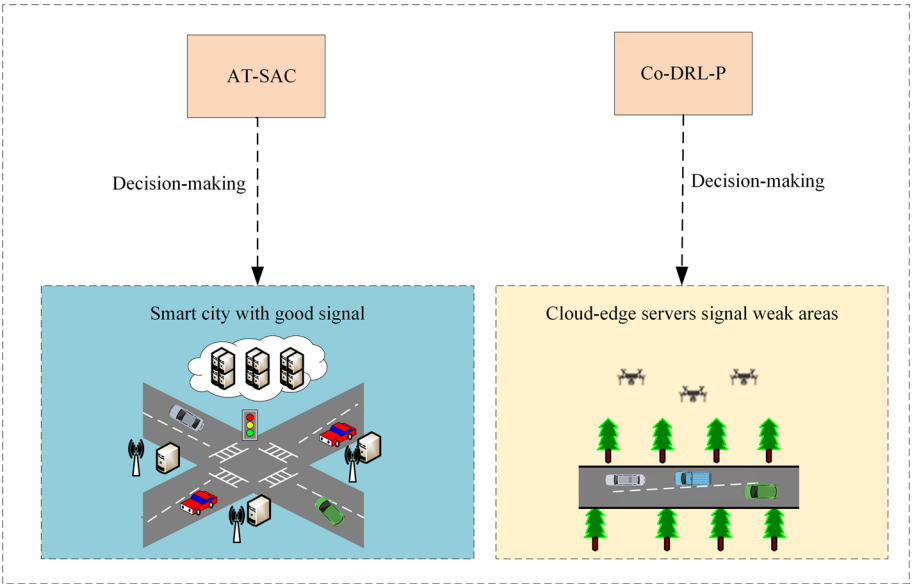


Fig. 4 Optimizing the synergistic process

empirical replay mechanism further reduces the estimation error (Xi et al. 2023). The flow of AT-SAC algorithm designed in this paper is shown in (Fig. 5).

In this paper, the objective optimization problem in cloud-edge-end collaborative decision-making is transformed into a Markov game process, and the problem is solved by using the AT-SAC through the interaction between the intelligent body and the intelligent traffic environment system. The state space, action space, state transfer probability and reward function are denoted by the quaternion $< \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} >$.

4.2.2 Adaptive temperature adjustment strategy based on entropy objective

The temperature update rule based on entropy target update strategy designed in this paper is shown in Eq. (28).

$$\alpha \leftarrow \alpha - \lambda \times \nabla_{\alpha} \left(\alpha \times \log \pi(a_t \mid s_t) + \mathcal{H}_{\text{target}} \right) \tag{28}$$

where α denotes the temperature coefficient, λ denotes the temperature learning rate used to control the speed of adjustment, $\mathcal{H}_{\text{target}}$ denotes the desired strategy entropy, and $\pi(a_t \mid s_t)$ denotes the probability distribution of the strategy.

4.2.3 State space

In the dynamic landscape of intelligent transportation, which is characterized by cloud-edge-end collaboration, the state space experiences continual evolution across various time slots. Agents forecast the state of the subsequent time slot by analyzing the system state observed in the current time slot. In this work, the execution of all the tasks of the system

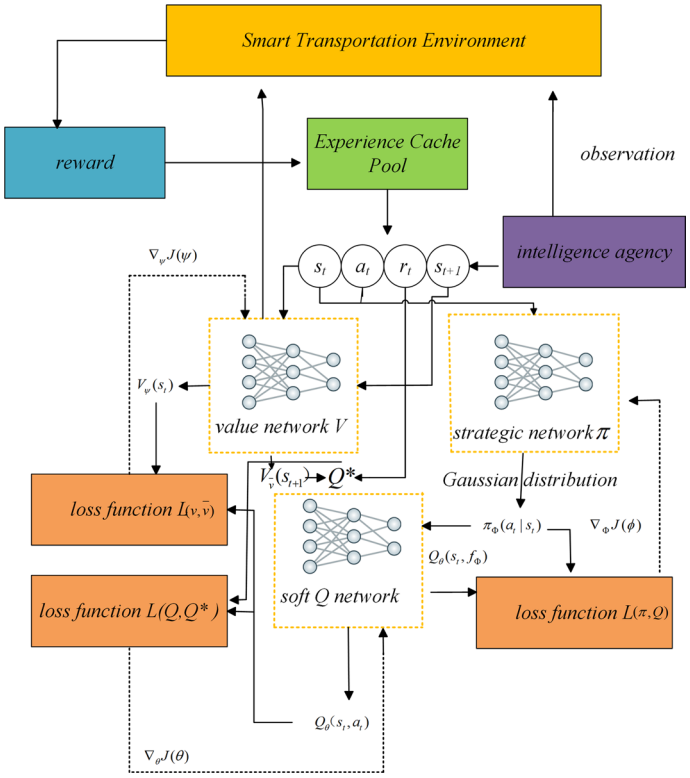


Fig. 5 AT-SAC algorithm flow

at the time slot of the vehicle-mounted terminal (ex_t), the task output state (ox_t), and the current task execution (α_t) are used as the system state. $\mathcal{S}$ is defined as the state space, and the state $s_t \in \mathcal{S}$ is shown in Eq. (29).

$$s_t = \{ex_t, ox_t, \alpha_t\} \tag{29}$$

4.2.4 Action space

The composition of the action space is as follows: the selection of the offloading decision target server (cr_t), the execution of the current task (tr_t), the input state of the task (ir_t). Define $\mathcal{A}$ as the action space and the action $a_t \in \mathcal{A}$, as shown in Eq. (30).

$$a_t = \{cr_t, tr_t, ir_t\} \tag{30}$$

4.2.5 Reward function

In this paper, we define the system reward value by considering the system delay, energy consumption, and resource loss resulting from task offloading by the vehicle-mounted terminal in the intelligent transportation cloud-edge-end collaboration system. Since different

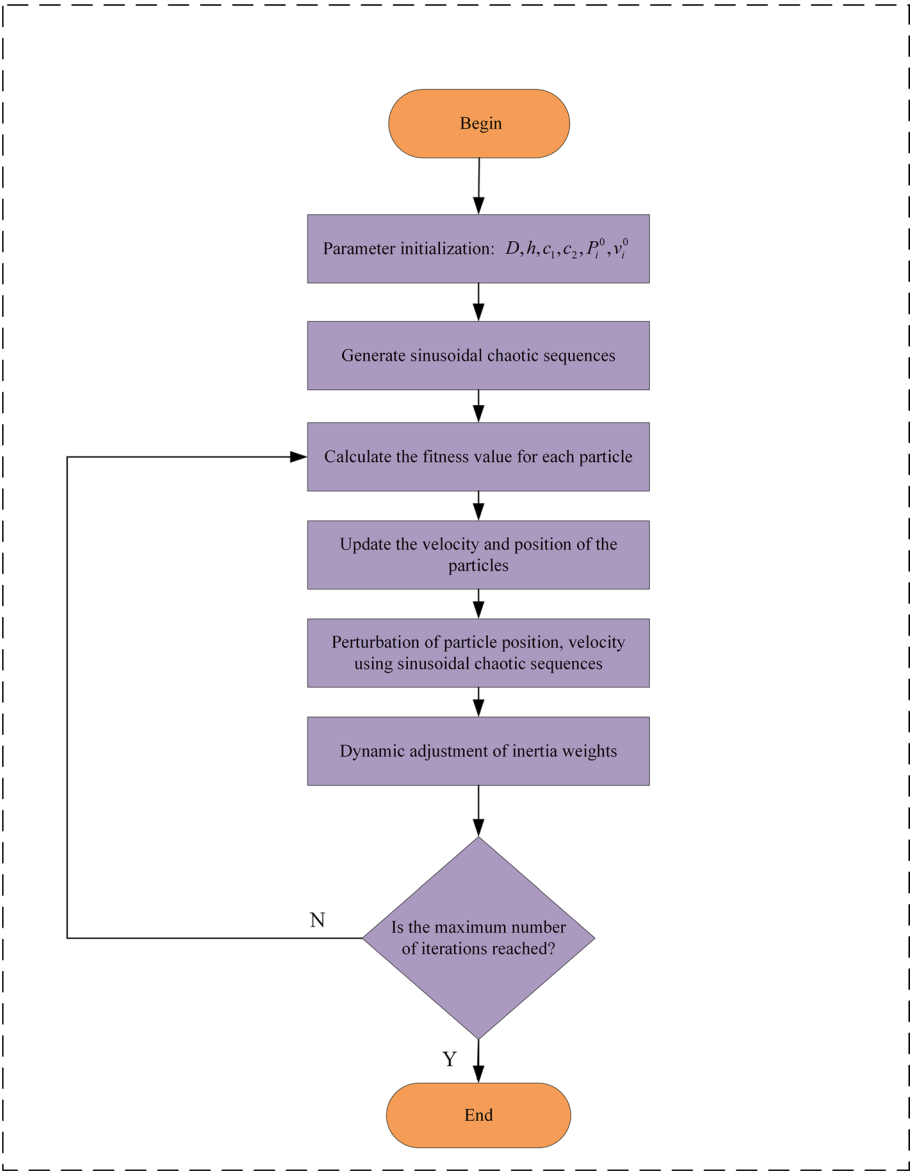


Fig. 6 A-PSO algorithm flow

user groups in the intelligent transportation system have different expectation optimization tendencies, this paper takes w_1, w_2, w_3 as the weight coefficient of system delay, energy consumption, and system resource loss. The reward function is shown in Eq. (31).

$$Reward_t = -(w_1 \times T_t(x) + w_2 \times E_t(x) + w_3 \times RL_t(x)) \tag{31}$$

In Eq. (31), $w_1 + w_2 + w_3 = 1$.

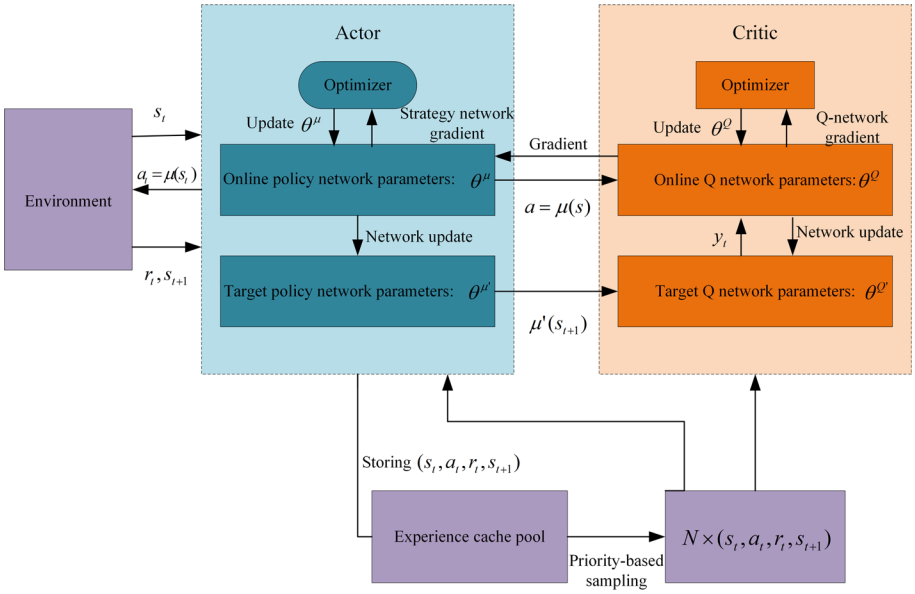


Fig. 7 A-PSO algorithm flow

4.2.6 Markov decision process

Based on Markov decision making, the objective function of the AT-SAC algorithm proposed in this paper is shown in Eq. (32).

$$J(\pi)=\mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty}\gamma^t\left(R(s_t,a_t)+\alpha\mathcal{H}(\pi(\cdot|s_t))\right)\right] \tag{32}$$

where $(\mathcal{H}(\pi(\cdot|s_t))=-\mathbb{E}_{a_t\sim\pi}\left[\log\pi(a_t|s_t)\right])$is the entropy of the strategy, indicating the randomness of the strategy output distribution, $\alpha(\alpha>0)$ is the entropy weighting factor used to balance exploration and reward.

The AT-SAC algorithm for updating the Q-function using soft targets $V(s)$ and Bellman's equation is shown in Eq. (33).

$$Q(s_t,a_t)=R(s_t,a_t)+\gamma\mathbb{E}_{s_{t+1}\sim\mathcal{P}}\left[V(s_{t+1})\right] \tag{33}$$

The soft target value function $V(s)$ is shown in Eq. (34).

$$V(s_t)=\mathbb{E}_{a_t\sim\pi}\left[Q(s_t,a_t)-\alpha\log\pi(a_t|s_t)\right] \tag{34}$$

The policy update for AT-SAC is shown in Eq. (35).

$$J_\pi=\mathbb{E}_{s_t\sim\mathcal{D},a_t\sim\pi}\left[\alpha\log\pi(a_t|s_t)-Q(s_t,a_t)\right] \tag{35}$$

4.2.7 AT-SAC algorithm programming

The AT-SAC algorithm program designed in this paper is shown in algorithm 1.

Input: Entropy target $\mathcal{H}_{\text{target}}$, discount factor γ, learning rates α_{actor}, α_{critic}, $\alpha_{\text{temperature}}$, soft update factor τ, batch size N, replay buffer $\mathcal{B}$, maximum iterations $\text{ite}_{\max}$
Output: Optimized policy $\pi^*(a|s)$
Initialize: Actor network $\pi_\theta(a|s)$, Critic networks Q_{ϕ_1}, Q_{ϕ_2} and their targets $Q_{\phi_1'}, Q_{\phi_2'}$, temperature parameter α, and replay buffer $\mathcal{B}$
for each iteration $t = 1$ to $\text{ite}_{\max}$ **do**
    Observe s_t, select action $a_t \sim \pi_\theta(a|s_t)$, execute a_t, observe r_t and s_{t+1}
    Store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{B}$
    if $\mathcal{B}$ contains at least N samples **then**
        Sample N transitions (s, a, r, s') from $\mathcal{B}$
        Compute target Q-value:

$$y = r + \gamma \cdot \left[\min_{i=1,2} Q_{\phi_i'}(s', a') - \alpha \cdot \log \pi_\theta(a'|s') \right]$$

        Update α: $\alpha \leftarrow \alpha + \alpha_{\text{temperature}} \cdot \Delta\mathcal{H}$, where $\Delta\mathcal{H} = -\mathbb{E}_{a\sim\pi_\theta}[\log \pi_\theta(a|s)] - \mathcal{H}_{\text{target}}$
        Update ϕ_1, ϕ_2 using Critic loss:

$$L_{\text{critic}} = \frac{1}{N} \sum \left(Q_{\phi_i}(s, a) - y \right)^2$$

        Update θ using Actor loss:

$$L_{\text{actor}} = \mathbb{E}_{s\sim\mathcal{B}} \left[\alpha \cdot \log \pi_\theta(a|s) - Q_{\phi_1}(s, a) \right]$$

        Soft update targets:

$$\phi_i' \leftarrow \tau\phi_i + (1 - \tau)\phi_i', \quad i = 1, 2$$

    end if
end for
Return: $\pi^*(a|s)$

Algorithm 1 AT-SAC Algorithm

4.2.8 AT-SAC algorithm time complexity analysis

The AT-SAC algorithm has neural network models such as a value function network, an objective value function network, and a policy function network. Typically, the complexity of a single neural network is related to the dimension of the input layer D_{in}, the dimension of the output layer D_{out}, the number of hidden layers, and the number of neurons in the hidden layers. Let L be the number of hidden layers and N_i be the number of neural units in the hidden layers, then, the complexity of a single neural network is $\mathcal{O}(D_{in} + \sum_{i=1}^{L} N_i + D_{out})$.

The number of loop iterations N_{epi} and the number of iterations of the algorithm for a single loop N_{it}, as well as the batch size B for each iteration also affect the complexity of the algorithm.

Therefore, the overall time complexity of the AT-SAC algorithm is shown in Eq. (36).

$$\mathcal{O}\left(3 \times \left(D_{in} + \sum_{i=1}^{L} N_i + D_{out}\right)(N_{epi} \times N_{it} \times B)\right) \tag{36}$$

4.2.9 AT-SAC algorithm space complexity analysis

The space complexity of the AT-SAC algorithm primarily depends on the parameters of the storage neural network and the experience replay buffer. Assuming that the input dimension of the neural network is D_{in}, the output dimension is D_{out}, the number of hidden layers is L, and the number of hidden units per layer is N_i, the space complexity for storing the parameters of a three-layer neural network in the AT-SAC algorithm is denoted by $\mathcal{O}(3 \times (D_{in} + \sum_{i=1}^{L} N_i + D_{out}))$. For the space complexity of storing the parameters of the experience playback pool, we assume that the dimension of the action is d_A, the dimension of the state is d_S, and the reward value is a scalar, then the total number of training rounds in the training process is N_{epi}, and the number of iterations per round is N_{it}. Then, for storing the parameters of the experience playback pool the space complexity is $\mathcal{O}(N_{epi} \times N_{it} \times (2(d_S) + d_A + 1))$.

Therefore, the overall space complexity of the AT-SAC algorithm is shown in Eq. (37).

$$\mathcal{O}\left(3 \times \left(D_{in} + \sum_{i=1}^{L} N_i + D_{out}\right) + (N_{epi} \times N_{it} \times (2(d_S) + d_A + 1))\right) \tag{37}$$

4.3 UAV-assisted offloading decision scheme

4.3.1 PSO algorithm design and improvement

The smart optimization algorithm PSO simulates the predatory behavior observed in birds, fish, and other social groups, offering distinctive advantages in tackling multi-objective optimization problems characterized by large scale and parallelism (Kumar De et al. 2023). The paper assumes that the server-equipped UAV terminal operates at a consistent altitude above the onboard users, delivering an assisted offloading service for the user terminals.

To adapt to the complex high-latitude optimization characteristics of intelligent transportation, this paper uses sinusoidal chaotic mapping to improve the particle swarm optimization algorithm. By leveraging the good distribution characteristics of chaotic sequences, the algorithm avoids falling into local optima, thereby enhancing the global search capability. This improved PSO algorithm is named A-PSO.

4.3.1.1 Particle position initialization based on chaotic mapping The initial position of the particle is set as shown in the Eq. (38).

$$P_i^0 = \sin(\pi \times x_{ki}) \tag{38}$$

where P_i^0 represents the initial position of particle *i*, and x_{ki} represents the current position of the particle.

4.3.1.2 Particle velocity initialization based on chaotic mapping The initial velocity of the particle is shown in the Eq. (39).

$$v_i^0 = \sin(\pi \times x_{ki}) \tag{39}$$

where v_i^0 is the initial position of particle *i*.

4.3.1.3 Particle velocity and position update based on chaotic mapping Let $P_{k_{\text{best}}}^h = (x_{k1}, x_{k2}, \ldots, x_{kD})$ be the historical optimal position of the particle and $G_{k_{\text{best}}}^h = (x_1, x_2, \ldots, x_D)$ be the global optimal position of the particle. The velocity and position of the UAV terminal in the feasible region are subsequently updated as shown in the following Eqs. (40)and(41).

$$v_k^{h+1} = \vartheta^h v_k^h + c_1 r_1 \left(P_{k_{\text{best}}}^h - x_k^h\right) + c_2 r_2 \left(G_{k_{\text{best}}}^h - x_k^h\right) + \sin(\pi \times x_k^h) \tag{40}$$

$$x_k^{h+1} = x_k^h + v_k^h + \sin(\pi \times x_k^h) \tag{41}$$

where *h* represents the current iteration number of the particle,ϑ represents the inertia weights, c_1, c_2 represent the individual and social learning factors, and r_1, r_2 represent random numbers between (0, 1).

4.3.1.4 Adaptive inertia weight update The inertia weights are updated as shown in the following Eq. (42).

$$g = g_{\max} - \frac{g_{\max} - g_{\min}}{H} \times \text{ite} \tag{42}$$

where *H* represents the total number of iterations of the particle, $g_{\max}$ is the initial inertia weight, $g_{\min}$ is the minimum inertia weight, ite is the current number of iterations.

4.3.1.5 A-PSO algorithm flow The A-PSO algorithm flow is shown in (Fig. 6).

4.3.2 DDPG algorithm design and improvement

The DDPG algorithm is commonly used to solve continuous action space problems. The introduction of an exploratory noise mechanism enables agents to dynamically adjust to environments characterized by noise and high-dimensional state spaces (Wang et al. 2023).

4.3.2.1 Priority-based experience playback mechanism In this paper, the original DDPG algorithm is improved by adding a priority-based experience playback mechanism, with the aim of ensuring that more valuable experiences are more likely to be sampled, thus accelerating the learning, keeping the rewards obtained by the intelligent body stable and comparable, and preventing large changes in rewards from negatively affecting the training. In this paper, we name the improved DDPG algorithm as ERDDPG and use it to optimize the communication and flight angle of the UAV terminal that acts as an intelligent body in the scenario. The communication and flight angle of the UAV terminal at the time slot of t_0 are taken as the system actions, and the auxiliary offloading situation of the task, the user position and the position of the UAV terminal are taken as the system states. Let the action space of the UAV terminal at the time slot of t be defined as $\{A_0 \mid a_t^1 = (\theta_t)\}$ and the state space be defined as $\{S_0 \mid s_t^1 = (D, L_t^{uav_k}, L_t^{user_m})\}$, the opposite of the objective function of $A - PSO$ is used as the reward value, and the objective is to maximize the reward.

4.3.2.2 ERDDPG algorithm flowchart The flow of ERDDPG algorithm designed in this paper is shown in (Fig. 7).

4.3.3 Co-DRL-P algorithm programming

In this paper, A-PSO and ERDDPG algorithm are combined to design Co-DRL-P algorithm to optimize the task scheduling, UAV communication angle and UAV group trajectory in ITS. The flow of Co-DRL-P algorithm designed in this paper is shown in algorithm 2.

Input:
  Particle number N, initial positions X_i and velocities V_i, fitness function $F(x)$, maximum iterations $T_{\max}$, state space S, action space A, reward function $r_t = -F(x_t)$
Output:
  Optimized task scheduling strategy X^*, UAV communication angles θ^*, UAV flight trajectories T^*
Begin:
for each particle i **do**
    Initialize X_i and V_i using a chaotic map
    Evaluate X_i and set $pBest_i \leftarrow X_i$
end for
$gBest \leftarrow \min\{pBest_i\}$
for each iteration t **do**
    A-PSO Phase:
    for each particle i **do**
        Update velocity and position:

$$v_i \leftarrow g \cdot v_i + c_1 \cdot \text{rand}() \cdot (pBest_i - X_i) + c_2 \cdot \text{rand}() \cdot (gBest - X_i)$$

$$X_i \leftarrow X_i + v_i$$

        Evaluate X_i and update $pBest_i$ and $gBest$
    end for
    ERDDPG Phase:
    Select action a_t using policy with exploration noise
    Execute action a_t, observe reward r_t and next state s_{t+1}
    Store (s_t, a_t, r_t, s_{t+1}) in replay buffer
    if buffer contains enough samples **then**
        Sample a batch, normalize rewards
        Compute Critic loss and update Critic network
        Update Actor network using policy gradient
        Perform soft update of target networks
    end if
end for
Output:
  Optimized X^*, θ^*, and T^*
End procedure

Algorithm 2 Co-DRL-P Algorithm

4.3.4 Co-DRL-P algorithm time complexity analysis

The time complexity of the Co-DRL-P optimization algorithm is primarily determined by the time complexities of the PSO algorithm and the ERDDPG algorithm, as well as the computational time involved in their interactions. For the PSO algorithm, assuming that the total number of tasks is *M*, the total number of particles is *N*, and the number of algorithm iterations is *T*, the time complexity of the PSO component is denoted by $\mathcal{O}(M \times N \times T)$. The time complexity of the ERDDPG algorithm is primarily influenced by the size of the experience replay pool, the batch size per training round, the number of updates per round,

and the number of parameters in the neural network. In the ERDDPG algorithm, assuming that the total number of training generations is N_{epi}, the number of updates per generation is N_{it}, the sample size of each batch is B, the size of the experience pool is M, and the number of neural network parameters is P, the time complexity of the ERDDPG algorithm is determined by $\mathcal{O}(N_{epi} \times N_{it} \times B \log M + B \times P)$.

Therefore, the overall time complexity of the Co-DRL-P algorithm is shown in Eq. (43).

$$\mathcal{O}(N \times T \times (M + N_{it} \times (B \log M + B \times P) \tag{43}$$

4.3.5 Co-DRL-P algorithm space complexity analysis

The space complexity of the Co-DRL-P optimization algorithm is primarily determined by the space complexities of the PSO and ERDDPG algorithms. In the particle swarm optimization algorithm, assuming that the particle dimension is D, the space complexity for storing the positions and velocities of N particles is $\mathcal{O}(N \times D)$. In the ERDDPG algorithm, we assume that the size of each sample is S,then the space complexity of the ERDDPG algorithm is $\mathcal{O}(M \times S + P)$. In the Co-DRL-P algorithm, the PSO and ERDDPG components share a common experience replay pool, and ERDDPG training is conducted following each particle update.

Consequently, the overall space complexity of the Co-DRL-P algorithm is expressed as shown in Eq. (44).

$$\mathcal{O}(N \times D + M \times S + P) \tag{44}$$

5 Experimentation and analysis

To demonstrate the effectiveness of the proposed strategy, we measure the scheme's performance using system latency, system energy consumption, and system resource loss during task offloading by in-vehicle users after applying the caching strategy. The simulation experiments designed in this paper are performed on an intel core i9-10900X cpu via pycharm simulation software in python 3.9 and the torch 2.11 environment. In order to prove the effectiveness of the proposed scheme in this paper, we conducted simulation experiments to compare the proposed optimization scheme with the latest optimization scheme of the existing research. we comprehensively compare the AT-SAC offloading scheme proposed in this paper with the deep Q-Learning offloading scheme (DQN) (Heidari and et al. 2022; Zhang et al. 2022), the DDPG scheme (Yang 2021), the twin delayed deep deterministic policy gradient(TD3) (Ye et al. 2024), and the collaborative randomized offloading scheme (Zhao 2024). In the cloud-edge-end collaboration scenario of UAV-assisted user offloading, we compare the scheme proposed in this paper (Co-DRL-P) with the PSO scheme (Kumar De et al. 2023), the NSGA-III scheme (Zhang et al. 2024) and the all-local scheme Zhu et al. (2025).

The details of the implementation of the baseline program and the specific workflow of the comparison in this paper are detailed below:

- AT-SAC scheme: the AT-SAC strategy proposed in this paper adopts a strategy gradient

deep reinforcement learning strategy based on three-layer neural network and a stabilized adaptive adjustment strategy based on entropy objective. Among them, the inclusion of dual Q-network effectively reduces the risk of overestimating the value function. By integrating the concept of maximum entropy reinforcement learning, the exploration ability of the agent in the learning process is enhanced and the stability of training is improved. The design of the entropy target-based stabilizing adaptive regulation strategy balances the exploration and utilization of the algorithm, thus better adapting to different training stages and task environments.

- DQN scheme: the reinforcement learning scheme that combines Q-learning with deep neural networks is used to enhance the stability of training by utilizing an empirical replay strategy and a target network to reduce the risk of strategy instability during training.
- DDPG scheme: the reinforcement learning algorithm based on the actor-critic architecture demonstrates effective learning in high-dimensional, continuous action spaces. By introducing deterministic strategies to eliminate randomness in policy selection, it accelerates convergence. Additionally, mechanisms such as experience replay and action exploration enable efficient exploration during training, enhancing sample efficiency.
- TD3 scheme: the deterministic deep reinforcement learning algorithm for solving continuous control problems, obtained by improving on the DDPG algorithm, can achieve excellent performance on many continuous control tasks by combining the deep deterministic policy gradient algorithm and double Q learning.
- Collaborative randomized scheme: in the cloud-edge-end collaboration framework, tasks are offloaded and computed, allowing vehicle terminals to randomly offload tasks to cloud servers, edge servers, or other vehicle terminals to fulfill service requests.
- Co-DRL-P scheme: A comprehensive optimization algorithm that integrates the PSO algorithm and the DDPG algorithm. By utilizing the powerful global search capability of PSO to effectively explore the complex search space and avoid local optimums, while DDPG utilizes the learning capability of policy and value networks for fine-grained local optimization. The combination of these two approaches compensates the limitations of PSO in complex dynamic environments and solves the problem of DDPG's dependence on gradient information.
- PSO scheme: the global optimization algorithm based on swarm intelligence mimics the collaborative behavior of flocks of birds or schools of fish in nature. It iteratively approaches the optimal solution of a problem by updating a population of particles within the search space. Each particle adjusts its position and velocity based on its own experience and the collective experience of the swarm.
- NSGA-III scheme: An improved multi-objective evolutionary algorithm is proposed for solving high-dimensional objective optimization problems. This algorithm significantly enhances the diversity and coverage of solutions by incorporating a reference point-based selection mechanism, a non-dominated sorting mechanism, and a complementary selection mechanism based on congestion distance. These enhancements enable the algorithm to exhibit excellent performance and robustness in addressing high-dimensional objective optimization challenges.
- All-local scheme: tasks are executed locally without the use of other servers.

5.1 Experimental parameters

This paper describes a dynamic three-dimensional spatial traffic scenario in which smart connected vehicles travel on the road at a speed of 20 *m*/*s* and UAV terminals fly over the road at a speed of 5 *m*/*s*. In order to simulate the dynamic network environment more realistically, this paper introduces a time-dependent first-order autoregressive model to simulate the small-scale fading channel gains of wireless communication links in different time slots. When simulation experiments are conducted, the user vehicle randomly performs structured tasks, and to prevent UAV terminals from colliding, this paper uniformly sets the UAV interval.

The key parameters of the algorithm in this paper are set as follows: the learning rate is 0.0001, the discount factor is 0.99, the batch size is 256, the number of neural network layers is 128, the number of neurons in each layer is 256, the optimizer uses Adam, the trade-off factors for latency, energy consumption, and resource depletion are 0.3 for w_1, 0.3 for w_2, and 0.4 for w_3. For the UAV-assisted offloading decision scheme, the trade-off coefficients for delay and energy consumption are of 0.5 and of 0.5. The configurations of the key parameters for the other simulation experiments involved in this paper are shown in Table 2 (Yu et al. 2023; Araf et al. 2023; Yang 2021; Zhu et al. 2025).

This section aims to empirically validate the effectiveness of the proposed cloud-side-end collaborative caching strategy, AT-SAC offloading scheme and UAV-assisted mission offloading scheme (Co-DRL-P). Among them, Fig. 8 illustrates the variation in system rewards of the proposed scheme under different hyperparameter settings. To demonstrate the robustness of the proposed optimization scheme in large-scale transportation systems with numerous users and tasks, Figs. 9 and 10 present a comparison of system latency and energy consumption across various optimization schemes for different numbers of users. Additionally, Figs. 11 and 12 provide a comparison of system latency and energy consumption for different task sizes. Finally, Fig. 13 highlights the contribution of each system component during the optimization process. In addition, to verify the effectiveness of the proposed UAV-assisted task offloading scheme (Co-DRL-P) in areas with weak cloud-side server signals, Fig. 14 presents a comparison of the system cost for each scheme across different application sizes. Finally, to further investigate the benefits of optimizing the UAV trajectory and communication angle for system users, Figs. 15 and 16 illustrate the on-board user task offloading and the system's communication range coverage after optimizing the UAV flight trajectory and communication angle.

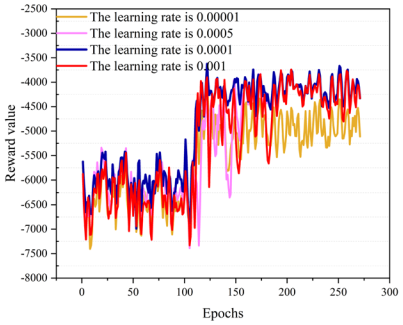
5.1.1 The Impact of CSEPCA caching policies on the performance of intelligent transportation systems

Figure 8 illustrates the reward values of the proposed scheme for various hyperparameters. Specifically, Fig. 8a depicts the reward convergence of the scheme across different learning rates. As observed from Fig. 8a, a larger learning rate accelerates the algorithm's convergence but introduces a higher degree of post-convergence oscillation, thereby compromising algorithmic stability. Conversely, a smaller learning rate hampers the algorithm's ability to reach the optimal solution. To balance optimal solution attainment and algorithmic stability, this study adopts a learning rate of 0.0001 for the subsequent simulation experiments. Figure 8b illustrates the reward convergence of the scheme across different batch sizes. As

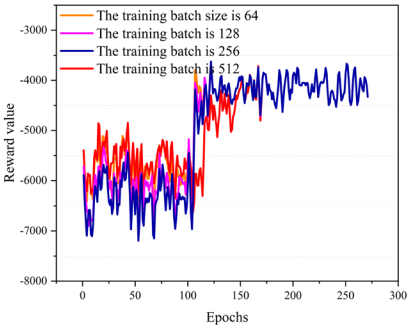
Table 2 Experimental parameters

Parameters	Notation	Numerical value
Amount of data of $task_i$	$d_{task_i}^{dav}$	50–70 MB
Required computing resources of $task_i$	$f_{task_i}^{calc}$	30–50 MIPS
Computing resources of cloud server	f_{cloud}^{calc}	1000 MIPS
Computing resources of ecd_n	$f_{ecd_n}^{calc}$	300–320 MIPS
Computing resources of car_m	$f_{car_m}^{calc}$	70–100 MIPS
Computing resources of uav_k	$f_{uav_k}^{calc}$	220–280 MIPS
Transmission power of cloud server	p_{cloud}^{tran}	80 dBm
Transmission power of ecd_n	$p_{ecd_n}^{tran}$	45 dBm
Transmission power of car_m	$p_{car_m}^{tran}$	25 dBm
Computational power of cloud server	p_{cloud}^{calc}	800 W
Computational power of ecd_n	$p_{ecd_n}^{calc}$	150–200 W
Computational power of car_m	$p_{car_m}^{calc}$	80 W
Computational power of uav_k	$p_{uav_k}^{calc}$	110-180 W
Caching resources of cloud server	s_{cloud}^{cach}	10,000 MHz
Caching resources of ecd_n	$s_{ecd_n}^{cach}$	3000 MHz
Caching resources of car_m	$s_{car_m}^{cach}$	800 MHz
UAV operating speed	$v_{uav_k}^{tran}$	5 m/s
Vehicle-mounted terminal operating speed	$v_{car_m}^{tran}$	20 m/s
Communication bandwidth for cloud server	B_0	150 MHz
Communication bandwidth for edge server	$B_{ecd_n}^{m,n}$	80 MHz
Communication bandwidth for UAV terminals	B_{uav}	80 MHz
Gaussian white noise power	θ^2	-70 dBm
Edge server communication radius	R_{ecd}	1.5 km
UAV communications radius	R_{uav}	100 m
Vehicle communication radius	R_{car}	10 m

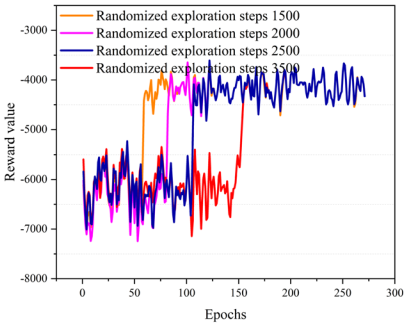
observed from Fig. 8b, larger batch sizes result in smoother and more stable reward changes during training; however, this stability comes at the cost of slower convergence. In contrast, smaller batch sizes can expedite convergence but are more prone to significant fluctuations during the later stages of training. To balance algorithmic stability and convergence speed, this study sets the batch size to 256 for subsequent simulation experiments. Figure 8c depicts the convergence of program rewards under different random exploration steps. As shown in Fig. 8c, a higher number of random exploration steps slows down the algorithm's convergence but provides a more extensive exploration space. Conversely, a lower number of random exploration steps accelerates convergence but may prevent the scheme from reaching the optimal solution. To balance the attainment of the optimal solution and the convergence speed of the rewards, this study sets the random step size to 2500 for the subsequent simulation experiments. Figure 8d illustrates the convergence of program rewards



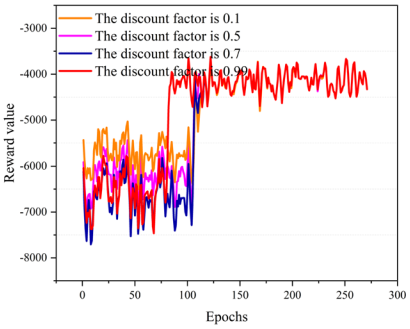
(a) The effect of different learning rates on reward values



(b) The effect of different batches sizes on reward values



(c) The effect of different randomized exploration steps on reward values



(d) The effect of different discount factors on reward values

Fig. 8 Training reward values

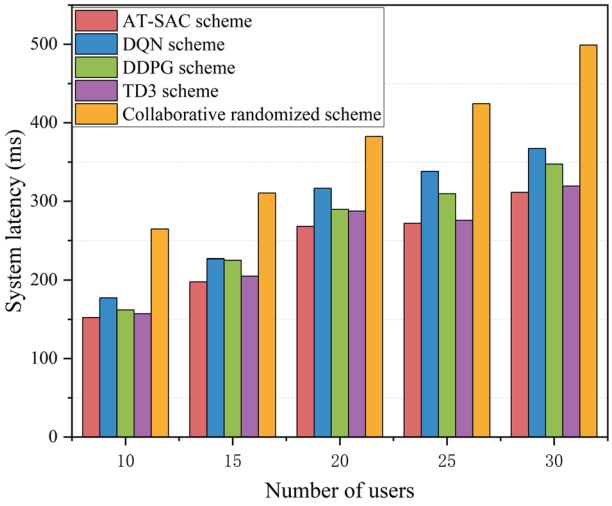


Fig. 9 Comparison of system delay consumption of each optimization scheme with different number of users

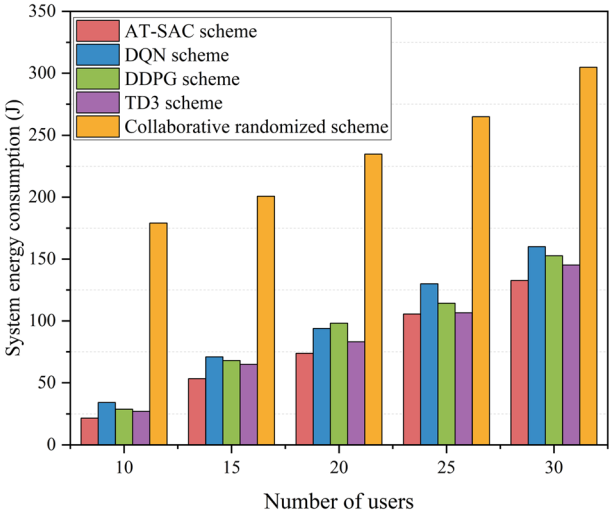


Fig. 10 Comparison of system energy consumption for each optimization scheme with different number of users

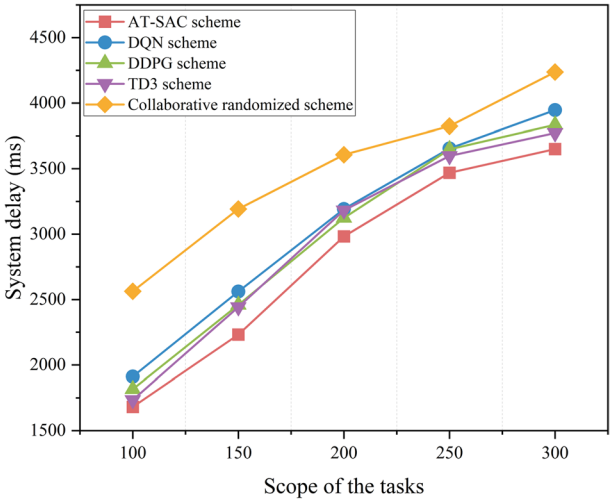


Fig. 11 Comparison of system delay consumption for each optimization scheme with different tasks sizes

under different discount factors. As shown in Fig. 8d, a smaller discount factor yields greater immediate rewards but makes it challenging to achieve higher long-term rewards. Conversely, a larger discount factor may limit immediate rewards but enables the scheme to attain better long-term rewards. Given that this study focuses on intelligent transportation cloud-edge-end collaboration scenarios, achieving higher long-term rewards is more beneficial for the system context considered. Therefore, the discount factor is set to 0.99 for the subsequent simulation experiments in this paper.

Figure 9 illustrates the comparison of system delay across various optimization schemes with different numbers of on-board users. As shown in Fig. 9, with the increase in the number of users in the intelligent transportation system, the total system delay for all schemes tends to increase. However, the proposed scheme consistently achieves the lowest total system delay. Notably, when the number of users reaches 30, the system delay of the proposed

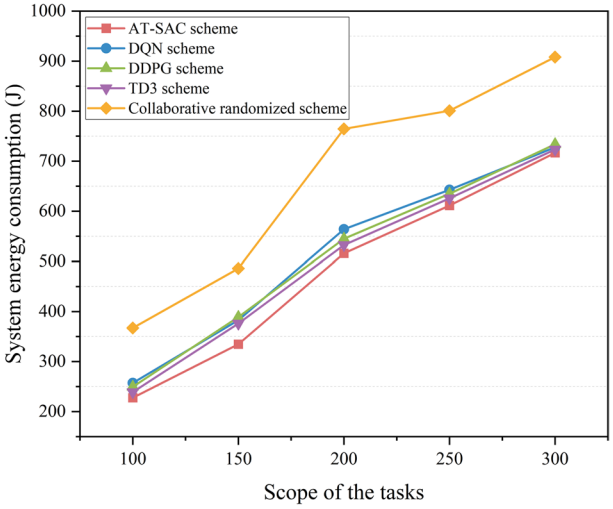
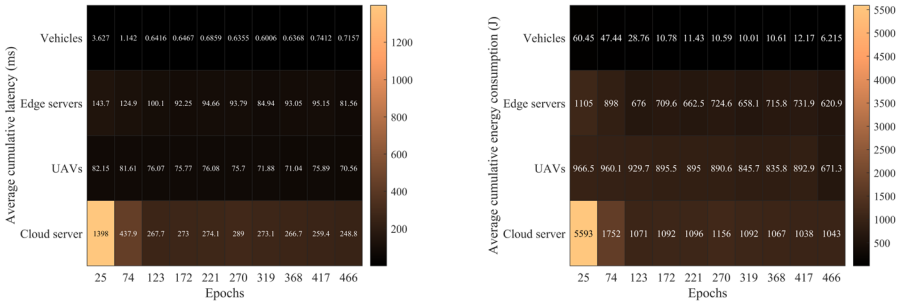


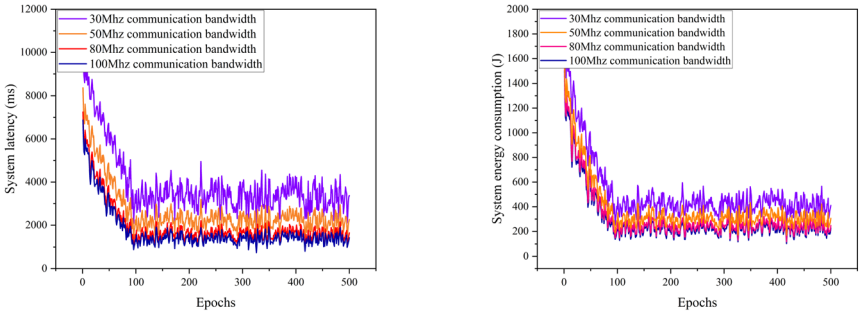
Fig. 12 Comparison of system energy consumption for each optimization scheme at different tasks sizes



(a) Heat map of average delay distribution by component

(b) Heat map of average energy consumption distribution by component

Fig. 13 The benefits of each component in the optimization process



(a) Impact of edge server and drone communication bandwidth on system network latency

(b) Impact of edge server and drone communication bandwidth on system energy consumption

Fig. 14 Impact of communication bandwidth on system performance

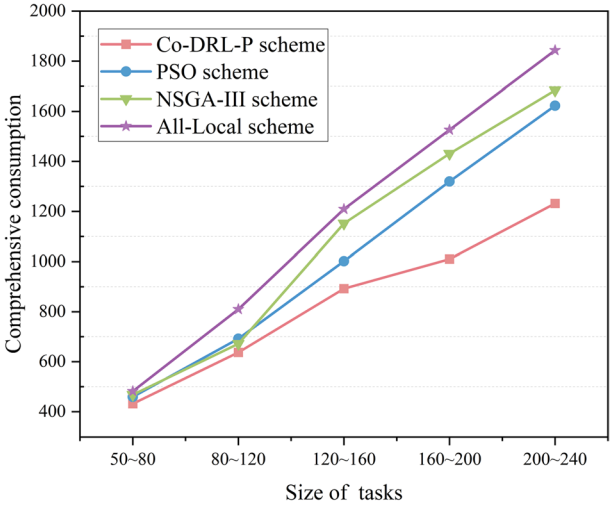


Fig. 15 Comprehensive system consumption comparison

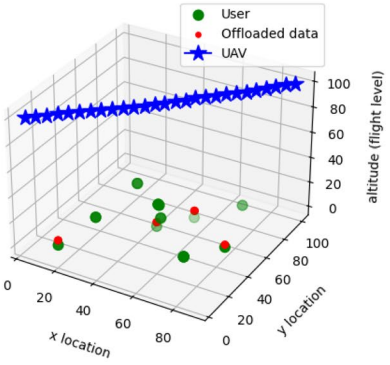
scheme is 17.9%, 11.5%, 2.6%, and 60.2% lower than that of the DQN, DDPG, TD3, and collaborative randomized schemes, respectively. These results demonstrate that the proposed scheme maintains strong performance even in large-scale transportation systems with a high number of users.

Figure 10 compares the system energy consumption across different optimization schemes with varying numbers of on-board users. As shown in Fig. 10, with an increase in the number of users in the intelligent transportation system, the system energy consumption for all schemes exhibits an upward trend. However, the proposed scheme consistently achieves the lowest energy consumption. Notably, when the number of users reaches 30, the system energy consumption of the proposed scheme is reduced by 20.6%, 15.9%, 9.4%, and 129.9% compared to the DQN, DDPG, TD3, and collaborative randomized schemes, respectively. These results indicate that the proposed scheme continues to perform effectively even in transportation systems with a large number of users.

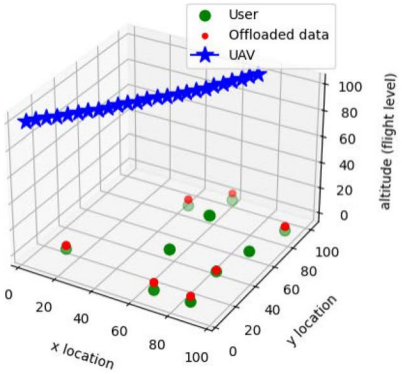
Figure 11 compares the system delay across different optimization schemes with varying numbers of application services. As shown in Fig. 11, as the number of application services in the intelligent transportation system increases, the system delay for all schemes tends to rise. However, the proposed scheme consistently achieves the lowest system delay. These results suggest that the optimization scheme presented in this paper can efficiently process tasks in transportation systems with a larger number of tasks, meeting the service demands of in-vehicle users.

Figure 12 compares the system energy consumption across different optimization schemes with varying numbers of application services. As shown in Fig. 12, as the number of application services in the intelligent transportation system increases, the system energy consumption for all schemes rises. However, the proposed scheme consistently achieves the lowest energy consumption. These results suggest that the optimization scheme presented in this paper continues to offer significant advantages in transportation systems with a larger number of tasks.

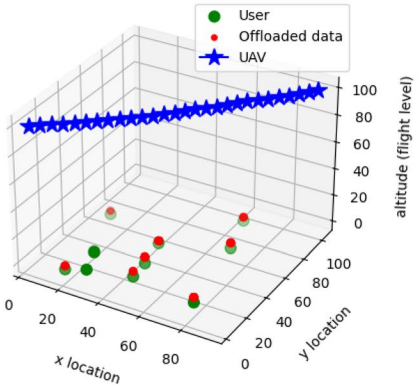
Figure 13 illustrates the performance gains of each component during the optimization process. As shown in Fig. 13a, b, both the latency consumption and energy consumption of



(a) 100 iterations of A-PSO and 20 rounds of ERDDPG training



(b) 100 iterations of A-PSO and 30 rounds of ERDDPG training



(c) 200 rounds of A-PSO iterations and 30 rounds of ERDDPG training

Fig. 16 D spatial comparison of UAV swarms assisting vehicle-mounted terminals in accomplishing mission offloading

each component exhibit a significant overall reduction, although minor fluctuations occur throughout the process. Notably, the cloud server achieves the greatest reduction in latency, with a decrease of approximately 4.6 times, while the in-vehicle terminal shows the largest reduction in energy consumption, with a decrease of approximately 9.4 times.

Figure 14 illustrates the impact of varying bandwidths of edge servers and UAVs on system latency and energy consumption. As shown in Fig. 14a and b, both system delay and energy consumption decrease as the server communication bandwidth increases. This is because an increase in server communication bandwidth results in a higher system throughput, which accelerates the data transmission rate. Consequently, this reduction in transmission delay and energy consumption leads to a decrease in both the total system delay and total energy consumption.

5.1.2 The impact of UAV-Assisted user task offloading on intelligent transportation system performance

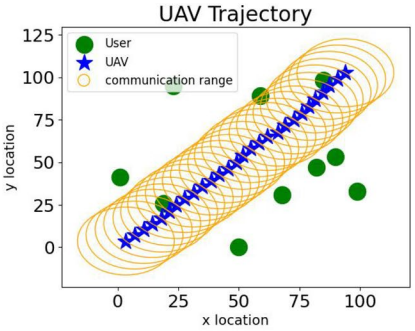
Figure 15 shows a comparison of the average integrated consumption of the system under the on-board local offloading scheme and the UAV-assisted scheme with different algorithms for various task sizes. Figure 15 clearly shows that the comprehensive cost of the system tends to increase as the task size increases. However, the comprehensive cost of each scheme using UAV-assisted task offloading consistently remains lower than the comprehensive cost consumed by on-vehicle local offloading. Furthermore, compared with other optimization algorithm schemes, the comprehensive cost of the proposed comprehensive optimization algorithm in this paper (Co-DRL-P) consistently ranks as the lowest.

Figure 16 provides a 3D spatial comparison of task offloading for UAV swarm-assisted vehicle terminals. A comparison of Fig. 16a and b clearly reveals that the number of UAV terminal-assisted vehicle offloading instances increases as the number of iterations of the ERDDPG algorithm increases. Specifically, at 100 rounds of A-PSO iterations and 20 rounds of ERDDPG training, the UAV terminal assisted 4 vehicle terminals to complete the offloading, and the number of assisted vehicle terminals to complete the offloading increased to 7 at 100 rounds of A-PSO iterations and 30 rounds of ERDDPG training (red labels in Fig. 16b). This observation indicates that the optimization achieves the desired effect. Furthermore, from the comparison of Fig. 16b and c, it can be concluded that under a fixed iteration number of the ERDDPG algorithm, the number of completions of offloading by the auxiliary vehicle terminal of the UAV increases with the increase in the iteration number of the A-PSO algorithm. This demonstrates that the strong global search capability of the A-PSO algorithm, combined with the robust continuous space processing capability of the ERDDPG algorithm, effectively assists the vehicle in completing task offloading.

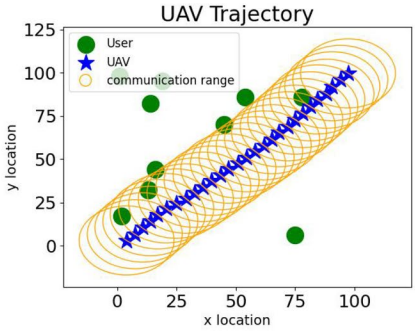
Figure 17 illustrates the optimization of the UAV running trajectory and communication angle. Figure 17 clearly shows that with optimization, the running trajectory of the UAV swarm aligns well with the driving route of the vehicle, ensuring that most user terminals fall within the effective communication range of the UAV. This indicates that the UAV can effectively learn valuable insights from the environment and adjust its actions accordingly, thereby optimizing the operation trajectory and communication angle. This observation underscores the effectiveness of the UAV-assisted offloading scheme proposed in this paper.

6 Conclusion and future work

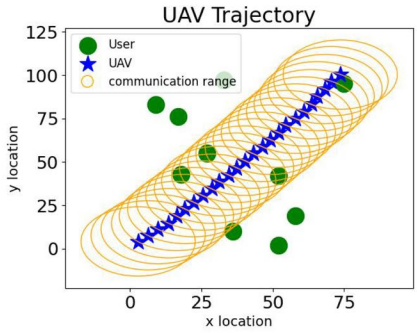
In this work, we address the specific application requirements of in-vehicle terminals in intelligent transportation scenarios and the resource characteristics of cloud servers. We propose a three-tier telematics caching strategy based on cloud-edge-end popularity cooperative allocation (CSEPCA). This strategy enables fine-grained replacement of cached content according to content popularity, optimizing both resource allocation and latency. To support the proposed strategy, we introduce several models, including those for communication, latency, energy consumption, and resource loss. Additionally, we present a multi-objective optimization model that employs the augmented deep reinforcement learning SAC algorithm (AT-SAC) to optimize system performance with respect to latency, energy consumption, and resource loss rates. In scenarios with weak cloud server signals, we design



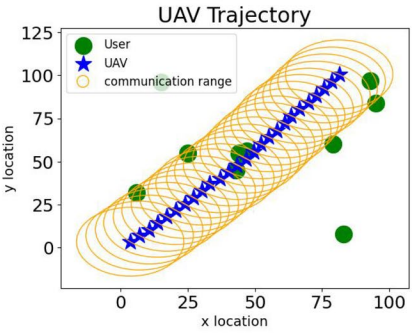
(a) 100 iterations of A-PSO and 30 rounds of ERDDPG training



(b) 100 iterations of A-PSO and 35 rounds of ERDDPG training



(c) 100 iterations of A-PSO and 40 rounds of ERDDPG training



(d) 150 iterations of A-PSO and 40 rounds of ERDDPG training

Fig. 17 Optimization of the UAV trajectory and communication angle

a UAV-assisted vehicle offloading decision scheme. This scheme utilizes the Co-DRL-P optimization algorithm, which integrates an enhanced deep reinforcement learning (DDPG) algorithm (ERDDPG) and an improved particle swarm optimization (A-PSO) algorithm. The goal is to optimize the UAV trajectory and communication angle to maximize the quality of service for users. The proposed scheme is evaluated through comprehensive simulation experiments. Specifically, when the number of users is 30, the system latency of the proposed scheme is 17.9%, 11.5%, 2.6%, and 60.2% lower than baseline schemes such as DQN, DDPG, TD3, and collaborative randomized schemes, and the system energy consumption is reduced by 20.6%, 15.9%, 9.4%, and 129.9%. Notably, the overall system cost for drone-assisted user offloading is reduced by approximately 49.6% in areas with weak cloud server signals.

6.1 Comparison with existing ITS solutions

While most traditional ITS systems rely on fixed communication infrastructure, our UAV-assisted solution enhances adaptability, particularly in areas with weak signal coverage. The Co-DRL-P algorithm, which combines A-PSO and ERDDPG, optimizes task offloading

efficiency and improves overall system performance, providing a significant advantage in real-time decision-making scenarios. This approach addresses the limitations of current ITS models, which often struggle to adapt to rapidly changing traffic conditions and communication environments.

6.2 Limitations of this study and directions for future work

While the simulation results are promising, the assumptions regarding channel fluctuations, UAV battery life, and the speeds of vehicles and UAVs in our study are idealized. These assumptions may not fully reflect real-world traffic scenarios, introducing limitations that need to be addressed in future work.In our future work, we aim to optimize task offloading and caching in dynamic urban traffic environments by incorporating varying signal strengths. Additionally, we plan to explore the scalability of our model to other smart domains, such as smart grids, automated distribution systems, and other intelligent networks that require efficient communication and task allocation. The goal is to enhance the practical feasibility and portability of our approach, making it more adaptable to a wide range of applications. In addition, we will explore resource scheduling and heterogeneous server co-caching in an integrated space-space-ground network using next-generation Internet technologies. By further refining these models and algorithms, we hope to enhance the robustness and adaptability of the system to ensure its practical feasibility and wide deployment in real-world intelligent transportation systems.

Author contributions Each author contributed uniquely to this study. ZHU Sifeng and SONG Zhaowei conceptualized the research, developed the methodology, conducted the investigations, and drafted the initial manuscript. HUANG Chang long curated the data, performed formal analyses, and contributed to software development. ZHU Hai created visualizations, validated the findings, and participated in manuscript editing. QIAO Rui provided resources, supervised the project, and acquired funding. All the authors reviewed and approved the final manuscript.

Funding This paper was supported by the Natural Science Foundation Project of China (62172457), Tianjin Natural Science Foundation Project (22JCZDJC00600), Tianjin Research Innovation Project for Postgraduate Students(2022SKYZ393), Henan Science and Technology Innovation Talent Project (23HASTIT029).

Data availability The data in the paper are real and all experiments can be reimplemented.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

Andreou A, Mavromoustakis CX, Batalla JM, Markakis EK, Mastorakis G (2023) Uav-assisted rsus for v2x connectivity using voronoi diagrams in 6g+ infrastructures. IEEE Trans Intell Transp Syst 24(12):15855–15865. https://doi.org/10.1109/TITS.2023.3273716

Araf S, Saha AS, Kazi SH et al (2023) Uav assisted cooperative caching on network edge using multi-agent actor-critic reinforcement learning. IEEE Trans Veh Technol 72(2):2322–2337

Chai F, Zhang Q, Yao H et al (2023) Joint multi-task offloading and resource allocation for mobile edge computing systems in satellite iot. IEEE Trans Veh Technol 72(6):7783–7795

Chen Z et al (2024) Uitde: a uav-assisted intelligent true data evaluation method for ubiquitous IoT systems in intelligent transportation of smart city. IEEE Trans Intell Transp Syst 25(8):9597–9607. https://doi.org/10.1109/TITS.2024.3373411

Chen Z, Zhang L, Wang X et al (2023) Cloud-edge collaboration task scheduling in cloud manufacturing: an attention-based deep reinforcement learning approach. Comput Ind Eng 177:109053

Chen C, Zeng Y, Li Y et al (2023) A multihop task offloading decision model in mec-enabled internet of vehicles. IEEE Internet Things J 10(4):3215–3230

Dai X (2023) Task co-offloading for d2d-assisted mobile edge computing in industrial internet of things. IEEE Trans Indus Inf 9(1):480–490

Dai X (2023) Task offloading for cloud-assisted fog computing with dynamic service caching in enterprise management systems. IEEE Trans Indus Inf 19(1):662–672

Dai Z, Zhang Q, Zhao L et al (2023) Cloud-edge computing technology-based internet of things system for smart classroom environment. Int J Emerg Technol Learn (Online) 18(8):110–124

De Kumar S, Banerjee A, Majumder K (2023) Coverage area maximization using mofac-ga-pso hybrid algorithm in energy efficient wsn design. IEEE Access 11:99901–99917

Deng X, Yin J, Guan P et al (2023) Smart delay-aware partial computing task offloading for multiuser industrial internet of things through edge computing. IEEE Internet Things J 10(4):2954–2966

Heidari A et al (2022) Deep q-learning technique for offloading offline/online computation in blockchain-enabled green iot-edge scenarios. Appl Sci 12(16):8232

Heidari A, Navimipour NJ, Unal M (2023) A secure intrusion detection platform using blockchain and radial basis function neural networks for internet of drones. IEEE Internet Things J 10(10):8445–8454

Heidari A, Navimipour NJ, Jamali MAJ et al (2023) A hybrid approach for latency and battery lifetime optimization in IoT devices through offloading and cnn learning. Sustain Comput Inf Syst 39:100899

Heidari A, Jamali MAJ, Navimipour NJ et al (2023) A qos-aware technique for computation offloading in iot-edge platforms using a convolutional neural network and markov decision process. IT Professional 25(1):24–39

Heidari A, Navimipour NJ, Jamali MAJ et al (2023) A green, secure, and deep intelligent method for dynamic IoT-edge-cloud offloading scenarios. Sustain Comput Inf Syst 38:100859

Hou C, Zhou C, Huang Q et al (2024) Cache control of edge computing system for tradeoff between delays and cache storage costs. IEEE Trans Autom Sci Eng 21(1):827–843

Jiang X, Dai X, Xiao Z et al (2023) Joint task offloading and resource allocation for energy-constrained mobile edge computing. IEEE Trans Mob Comput 22(7):4000–4015

Li Y (2023) Collaborative content caching and task offloading in multi-access edge computing. IEEE Trans Veh Technol 72(4):5367–5372

Liu RW, Guo Y, Lu Y et al (2023) Deep network-enabled haze visibility enhancement for visual IoT-driven intelligent transportation systems. IEEE Trans Indus Inf 19(2):1581–1591

Long S, Zhang Y, Deng Q et al (2023) An efficient task offloading approach based on multi-objective evolutionary algorithm in cloud-edge collaborative environment. IEEE Trans Netw Sci Eng 10(2):645–657

Lu W (2023) Secure transmission for multi-uav-assisted mobile edge computing based on reinforcement learning. IEEE Trans Netw Sci Eng 10(3):1270–1282

Luo L, Sun R, Chai R et al (2023) Cost-efficient uav deployment and content placement for cellular systems with d2d communications. IEEE Syst J 17(4):5405–5416

Miao J, Wang Z, Ning X, Shankar A, Maple C, Rodrigues JJPC (2024) A uav-assisted authentication protocol for internet of vehicles. IEEE Trans Intell Transp Syst 25(8):10286–10297. https://doi.org/10.1109/TITS.2024.3360251

Peng P, Lin W, Wu W et al (2024) A survey on computation offloading in edge systems: from the perspective of deep reinforcement learning approaches. Comput Sci Rev 53:100656

Samanta A, Nguyen TG, Ha T et al (2023) Distributed resource distribution and offloading for resource-agnostic microservices in industrial IoT. IEEE Trans Veh Technol 7(21):1184–1195

Tian A (2023) Efficient federated drl-based cooperative caching for mobile edge networks. IEEE Trans Netw Serv Manage 20(1):246–260

Tian A (2023) Efficient federated drl-based cooperative caching for mobile edge networks. IEEE Trans Netw Serv Manage 20(1):246–260
Wang T, Lu Y, Wang J et al (2021) Eihdp: edge-smart hierarchical dynamic pricing based on cloud-edge-client collaboration for iot systems. IEEE Trans Comput 70(8):1285–1298
Wang Z, Liu R, Liu Q et al (2023) Qos-oriented sensing-communication-control co-design for uav-enabled positioning. IEEE Trans Green Commun Netw 7(1):497–511
Wang J, Wang Y, Cheng P et al (2023) Ddpg-based joint resource management for latency minimization in noma-mec network. IEEE Commun Lett 27(7):1814–1818
Wu D (2024) Uav-assisted real-time video transmission for vehicles: a soft actor-critic drl approach. IEEE Internet Things J 11(8):14710–14726. https://doi.org/10.1109/JIOT.2023.3343590
Wu H, Lin W, Shen W, et al (2024) Prediction of heterogeneous device task runtime based on edge server-oriented deep neuro-fuzzy system. IEEE Trans Serv Comput
Wu H, Shen W, Lin W, et al (2024) End-edge-cloud heterogeneous resources scheduling method based on rnn and particle swarm optimization. IEEE Trans Netw Ser Manage
Xi F, Ruan Y, Li Y (2023) Soft actor-critic based 3-d deployment and power allocation in cell-free unmanned aerial vehicle networks. IEEE Wireless Commun Lett 12(10):1692–1696
Xia Y, Zhang H, Zhou X et al (2023) Location-aware and delay-minimizing task offloading in vehicular edge computing networks. IEEE Trans Veh Technol 72(12):16266–16279
Xiao F, Yu S, Li Y (2023) Efficient large-capacity caching in cloud storage using skip-gram-based file correlation analysis. IEEE Access 11:111265–111273
Yan SR, Pirooznia S, Heidari A et al (2022) Implementation of a product-recommender system in an IoT-based smart shopping using fuzzy logic and apriori algorithm. IEEE Trans Eng Manage 71:4940–4954
Yan X, Hu Y, Zhang J et al (2023) Joint user scheduling and uav trajectory design on completion time minimization for uav-aided data collection. IEEE Trans Wireless Commun 22(6):3884–3898
Yang L (2021) On the application of cooperative noma to spatially random wireless caching networks. IEEE Trans Veh Technol 70(11):12055–12071
Yang G, Yao Y (2023) Resource allocation control of uav-assisted IoT communication device. IEEE Trans Intell Transp Syst 24(11):13341–13349. https://doi.org/10.1109/TITS.2022.3219048
Ye J, Guo H, Zhao D, Wang B, Zhang X (2024) Td3 algorithm based reinforcement learning control for multiple-input multiple-output dc-dc converters. IEEE Trans Power Electron 39(10):12729–12742. https://doi.org/10.1109/TPEL.2024.3416911
Yin C, Yang H, Xiao P et al (2023) Resource allocation for uav-assisted wireless powered d2d networks with flying and ground eavesdropping. IEEE Commun Lett 27(8):2103–2109
Yu L, Liu Z, Fan R et al (2023) Optimal computation offloading in collaborative leo-IoT enabled mec: a multiagent deep reinforcement learning approach. IEEE Trans Green Commun Netw 7(2):996–1011
Yuan Y, Sun E, Qu H (2023) Joint multi-ground-user edge caching resource allocation for cache-enabled high-low-altitude-platforms integrated network. IEEE Trans Signal Inf Process Netw 9:655–668
Zeng F, Zhang K, Wu L et al (2023) Efficient caching in vehicular edge computing based on edge-cloud collaboration. IEEE Trans Veh Technol 72(2):2468–2481
Zhang Y (2022) Toward hit-interruption tradeoff in vehicular edge caching: Algorithm and analysis. IEEE Trans Intell Transp Syst 23(6):5198–5210
Zhang QZ, Min G et al (2022) Cooperative edge caching based on temporal convolutional networks. IEEE Trans Parallel Distrib Syst 33(9):2093–2105
Zhang Z, Chen Z, Shen Y et al (2024) A dynamic task offloading scheme based on location forecasting for mobile smart vehicles. IEEE Trans Veh Technol 73(6):7532–7546. https://doi.org/10.1109/TVT.2024.3351224
Zhao L (2024) Meson: a mobility-aware dependent task offloading scheme for urban vehicular edge computings. IEEE Trans Mob Comput 23(5):4259–4272
Zhao H, Sun W, Ni Y, Xia W, Gui G, Zhu C (2024) Deep deterministic policy gradient-based rate maximization for ris-uav-assisted vehicular communication networks. IEEE Trans Intell Transp Syst 25(11):15732–15744. https://doi.org/10.1109/TITS.2024.3440391
Zhou F, Yang Q, Zhong T et al (2021) Variational graph neural networks for road traffic prediction in intelligent transportation systems. IEEE Trans Indus Inf 17(4):2802–2812
Zhou H, Jiang K, He S (2023) Distributed deep multi-agent reinforcement learning for cooperative edge caching in internet-of-vehicles. IEEE Trans Wireless Commun 22(12):9595–9609
Zhu S, Song Z, Huang C et al (2025) Dependency-aware cache optimization and offloading strategies for intelligent transportation systems. J Supercomput. https://doi.org/10.1007/s11227-024-06596-7
Zhu J, Wang X, Huang H et al (2022) A nsga-ii algorithm for task scheduling in uav-enabled mec system. IEEE Trans Intell Transp Syst 23(7):9414–9429

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Sifeng Zhu[1] · Zhaowei Song[1] · Changlong Huang[1] · Rui Qiao[2] · Hai Zhu[3]

✉ Hai Zhu
zhu_sea@163.com

Sifeng Zhu
zhusifeng@tcu.edu.cn

Zhaowei Song
szw9992022@163.com

Changlong Huang
xiaoguagua20@163.com

Rui Qiao
jorui_314@126.com

[1] School of Computer and Information Engineering, Tianjin Chengjian University, Tianjin 300384, China

[2] School of Computer, Zhoukou Normal University, Zhoukou 466001, Henan, China

[3] School of Computer, Henan University of Engineering, Zhengzhou 451191, Henan, China